

# 广州华立学院

## 本科生毕业论文（设计）

题目：基于大数据技术的个性化音乐推荐系统设计与实现

|        |               |
|--------|---------------|
| 学    院 | 计算机工程学院       |
| 专    业 | 数据科学与大数据技术    |
| 班    级 | Z1 大数据        |
| 学    号 | 5124440106796 |
| 姓    名 | 卢伟军           |
| 指导教师   | 王波            |

2026 年 4 月

## 原创性声明

本人郑重声明：在广州华立学院攻读学士学位期间，所提交的毕业论文（设计）《基于大数据技术的个性化音乐推荐系统设计与实现》，是本人在导师指导下独立进行研究工作所取得的成果。对本文的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明，其它未注明部分不包含他人已发表或撰写过的研究成果，不存在购买、由他人代写、剽窃和伪造数据等作假行为。

本人愿为此声明承担法律责任。

作者签名：卢伟军.

日期： 2026 年 4 月 20 日

## 摘 要

随着数字音乐流媒体平台用户规模与曲库容量的持续扩张，信息过载问题日益制约用户音乐消费体验。音乐消费行为的隐式性、用户兴趣的动态迁移以及长尾曲目的交互稀疏，加剧了个性化推荐的建模难度。本文基于 KKBOX 公开音乐数据集，设计并实现了一套个性化音乐推荐系统原型，旨在从海量候选曲目中精准匹配用户潜在偏好，并兼顾推荐结果的流派多样性。

系统由算法引擎 MusicMode 与用户交互平台 MusicWeb 两大模块构成。特征工程采用折外目标编码（OOF-TE）策略消除高基数类别特征的标签泄漏风险，构建涵盖行为统计、歌曲属性与协同过滤嵌入的多维特征集。推荐算法采用四层管道结构：召回层通过协同过滤、向量检索与热度兜底三路并行策略产生约 600 首候选曲，经 BST 行为序列 Transformer 粗排压缩至前 300 首。精排层以 DeepFM 与 LightGBM 并行建模，经 5 折 OOF Stacking 元学习器融合后截取前 150 首。重排层引入最大边际相关算法（MMR）调节流派多样性，最终为每位用户输出 50 首个性化推荐。MusicWeb 基于 Java Web MVC 架构，实现推荐展示、行为反馈采集与音乐检索等功能，通过隐式反馈闭环持续迭代推荐质量。

离线评估表明，Stacking 集成精排 AUC 达 0.8304，优于各单模型基线。消融实验逐层验证了四层管道各环节的正向增益，证明分层设计的合理性与有效性。本研究的主要创新在于提出基于 5 折 OOF Stacking 框架的异构集成精排策略，有效降低了稀疏交互数据下的过拟合风险。在管道末端引入 MMR 多样性重排，系统性缓解了音乐推荐中的信息茧房问题，为个性化音乐推荐提供了兼顾精度与多样性的完整工程实践方案。

**关键词：**大数据技术；个性化推荐；深度学习；集成学习；协同过滤；特征工程

## Abstract

With the rapid expansion of digital music streaming platforms in both user base and catalog size, information overload increasingly constrains the music consumption experience. The implicit nature of music consumption behavior, dynamic shifts in user preferences, and interaction sparsity among long-tail tracks further complicate personalized recommendation modeling. Based on the public KKBOX dataset, this paper presents the design and implementation of a personalized music recommendation system prototype, aimed at precisely matching users' latent preferences from massive candidate tracks while maintaining genre diversity in recommendation results.

The system consists of two modules: MusicMode, an algorithm engine, and MusicWeb, a user interaction platform. In feature engineering, an out-of-fold target encoding (OOF-TE) strategy is applied to eliminate label leakage in high-cardinality categorical features, constructing a multi-dimensional feature set that covers user behavior statistics, track attributes, and collaborative filtering embeddings. The recommendation algorithm adopts a four-stage pipeline: the recall stage generates approximately 600 candidates through three parallel channels—collaborative filtering, vector retrieval, and popularity fallback—which are then compressed to 300 by a Behavior Sequence Transformer (BST) in pre-ranking. In fine-ranking, DeepFM and LightGBM perform parallel modeling and are fused by a 5-fold OOF Stacking meta-learner, retaining the top 150 candidates. A Maximal Marginal Relevance (MMR) algorithm then re-ranks results to regulate genre diversity, ultimately delivering 50 personalized tracks per user. MusicWeb is built on a Java Web MVC architecture, implementing recommendation display, behavioral feedback collection, and music search, with an implicit feedback loop continuously refining recommendation quality.

Offline evaluation shows that the Stacking ensemble fine-ranker achieves an AUC of 0.8304, outperforming each individual model baseline. Ablation experiments validate the positive contribution of each pipeline stage, confirming the rationality of the layered design. The primary innovations of this research include: a heterogeneous ensemble fine-ranking strategy based on a 5-fold OOF Stacking framework, which effectively reduces overfitting risk under sparse interaction data; and deployment of MMR diversity re-ranking at the pipeline endpoint, systematically alleviating the filter bubble problem in music recommendation and providing a complete engineering solution that balances recommendation accuracy with list diversity.

**Keywords:** big data technology, personalized recommendation, deep learning, ensemble learning, collaborative filtering, feature engineering

# 目 录

|                          |    |
|--------------------------|----|
| 摘 要 .....                | I  |
| Abstract.....            | II |
| 1 绪 论 .....              | 1  |
| 1.1 研究背景与意义 .....        | 1  |
| 1.2 国内外研究现状 .....        | 1  |
| 1.2.1 国内研究现状 .....       | 1  |
| 1.2.2 国外研究现状 .....       | 2  |
| 1.3 论文研究内容 .....         | 3  |
| 1.4 论文组织结构 .....         | 4  |
| 2 系统开发技术与算法选型 .....      | 6  |
| 2.1 大数据处理与召回技术 .....     | 6  |
| 2.2 核心推荐算法选型 .....       | 6  |
| 2.3 Java Web 核心技术栈 ..... | 7  |
| 2.4 本章小结 .....           | 8  |
| 3 系统需求分析 .....           | 9  |
| 3.1 系统总体目标 .....         | 9  |
| 3.2 系统用例分析 .....         | 10 |
| 3.2.1 参与者识别 .....        | 10 |
| 3.2.2 核心用例图 .....        | 10 |
| 3.2.3 推荐引擎端用例 .....      | 12 |
| 3.3 系统性能与安全性需求分析 .....   | 12 |
| 3.3.1 性能需求 .....         | 12 |
| 3.3.2 安全性需求 .....        | 13 |
| 3.4 本章小结 .....           | 13 |
| 4 系统设计 .....             | 15 |

|       |                          |    |
|-------|--------------------------|----|
| 4.1   | 系统总体架构设计 .....           | 15 |
| 4.1.1 | MusicWeb 前端模块架构 .....    | 15 |
| 4.1.2 | MusicMode 推荐引擎模块架构 ..... | 17 |
| 4.1.3 | 双模块交互与部署设计 .....         | 18 |
| 4.2   | 前端交互设计 .....             | 18 |
| 4.2.1 | 页面总体结构设计 .....           | 18 |
| 4.2.2 | 核心功能页面设计 .....           | 19 |
| 4.2.3 | 推荐交互与行为反馈设计 .....        | 20 |
| 4.3   | 推荐算法流水线设计 .....          | 20 |
| 4.3.1 | 召回层设计 .....              | 21 |
| 4.3.2 | 排序层设计 .....              | 22 |
| 4.3.3 | 重排层设计 .....              | 23 |
| 4.4   | 数据库设计 .....              | 24 |
| 4.4.1 | 数据库结构设计 .....            | 24 |
| 4.5   | 本章小结 .....               | 25 |
| 5     | 系统实现 .....               | 26 |
| 5.1   | 开发环境与工具配置 .....          | 26 |
| 5.2   | 数据处理与特征工程实现 .....        | 26 |
| 5.2.1 | 数据介绍与 ETL 处理 .....       | 26 |
| 5.2.2 | 数据清洗 .....               | 27 |
| 5.2.3 | 特征工程 .....               | 28 |
| 5.2.4 | 数据集时序切分 .....            | 30 |
| 5.3   | MusicMode 推荐算法实现 .....   | 31 |
| 5.3.1 | ALS 协同过滤召回实现 .....       | 31 |
| 5.3.2 | FAISS 向量检索召回实现 .....     | 31 |
| 5.3.3 | 热度兜底召回实现 .....           | 32 |

|            |                                 |    |
|------------|---------------------------------|----|
| 5.3.4      | BST 行为序列 Transformer 粗排实现 ..... | 32 |
| 5.3.5      | LightGBM 精排实现 .....             | 34 |
| 5.3.6      | DeepFM 深度精排实现 .....             | 35 |
| 5.3.7      | Stacking 集成与重排输出的实现 .....       | 36 |
| 5.4        | MusicWeb 前端实现 .....             | 38 |
| 5.4.1      | MVC 分层架构搭建 .....                | 39 |
| 5.4.2      | 用户认证与账户管理实现 .....               | 39 |
| 5.4.3      | 个性化推荐展示与行为反馈实现 .....            | 42 |
| 5.4.4      | 多源音乐检索与在线播放实现 .....             | 43 |
| 5.4.5      | 歌单管理功能实现 .....                  | 44 |
| 5.4.6      | 管理员后台功能实现 .....                 | 45 |
| 5.5        | 本章小结 .....                      | 48 |
| 6          | 系统测试与评估 .....                   | 49 |
| 6.1        | 评估指标与实验设置 .....                 | 49 |
| 6.1.1      | 评估指标说明 .....                    | 49 |
| 6.1.2      | 实验设置 .....                      | 51 |
| 6.2        | 消融实验结果分析 .....                  | 52 |
| 6.3        | 各模型 AUC 对比实验 .....              | 54 |
| 6.4        | 本章小结 .....                      | 56 |
| 结 论 .....  |                                 | 57 |
| 参考文献 ..... |                                 | 59 |
| 致 谢 .....  |                                 | 61 |
| 附 录 .....  |                                 | 63 |

# 1 绪 论

## 1.1 研究背景与意义

随着数字音乐流媒体平台规模的不断扩张，以 Spotify、网易云音乐为代表的主流平台曲库数量都达到了亿首量级，用户通过人工榜单或者搜索关键词来发现音乐的效率极低，信息过载问题日益突出。以及音乐推荐面临着用户偏好不一、跳曲、完播率等隐式行为和正负样本标注噪声较大的问题。通过对用户历史行为建模，从海量曲目中筛选出少量高潜偏好结果，是平台维持用户活跃的核心技术。此外，音乐消费具有极强的情境依赖性，静态偏好模型难以捕捉到用户兴趣的动态变化，而长尾曲目交互记录极为稀疏，协同过滤方法在冷启动场景下性能退化严重。

上述问题需要推荐系统在算法设计与工程实现两个维度都进行系统化处理，本文将基于 KKBOX 公开数据集设计并实现 MusicWeb 与 MusicMode 构成的双模块个性化音乐推荐系统，探索大数据处理与多模型异构精排在离线音乐场景下的工程集成，为中小型在线音乐平台提供参考。

## 1.2 国内外研究现状

### 1.2.1 国内研究现状

在工程化实践方向，赵吉<sup>[1]</sup>基于分布式计算框架构建了以矩阵分解协同过滤为核心的大规模音乐推荐系统，验证了分布式计算路径的工程可行性，但未引入深度学习精排层，对特征交叉信息的利用较为有限。杨率<sup>[2]</sup>将深度神经网络应用于个性化实时音乐推荐，探索了用户行为序列的动态偏好建模，然而其系统缺乏完整的前端交互平台，推荐结果的用户反馈闭环有待完善。上述两项工作初步论证了大数据技术在音乐推荐场景中的适用性，但均未形成召回、精排与重排的多层流水线，本文在此基础上引入多模型集成精排管道，以期在推荐精度与系统完整性上有所推进。

在语义增强与情感特征方向，李津<sup>[3]</sup>引入知识图谱丰富物品侧语义表示，对缓解



稀疏场景下的冷启动问题有一定效果，但图卷积的计算复杂度较高，难以适配百万量级曲库的实时推理需求。冯亚寒<sup>[4]</sup>提出了融合小波散射变换与多尺度卷积网络的音乐多情感分类方法，在情感特征提取上具有一定新颖性，但该方法聚焦于内容侧标签预测，未与用户行为数据联合建模，难以直接用于个性化推荐。吴亚迪和陈平华<sup>[5]</sup>设计了融合用户长短期偏好与情感注意力的推荐模型，在个性化偏好建模方面有所改进，但两者均依赖显式情感标注，在以隐式行为反馈为主的大规模数据集上的适用性有待进一步验证。

在综述与前沿方法方向，王慧心和童向荣<sup>[6]</sup>梳理了融合知识图谱的推荐系统研究进展，指出语义增强对于缓解数据稀疏性具有较好效果，但计算开销制约了其在实时推荐中的落地。梁锋等<sup>[7]</sup>对基于联邦学习的推荐系统隐私保护机制进行了综述，为在不共享原始数据条件下的协同建模提供了参考思路。郑楠等<sup>[8]</sup>对基于深度学习的群组推荐方法进行了分类比较，总结了不同群体偏好聚合策略的适用边界。闫猛等<sup>[9]</sup>提出了基于自监督学习的序列推荐算法，在减少对人工标注数据依赖方面具有一定借鉴价值。龚雪鸾等<sup>[10]</sup>对点击率预测方法进行了系统综述，梳理了从浅层模型到深度模型的演进路径，为本文精排层的模型选型提供了参考依据。上述综述类研究揭示了现有方法在计算效率、隐式反馈利用及多阶特征交叉等方面的共性不足，本文以 BST 承担粗排初筛职责，在精排层部署 DeepFM 与 LightGBM 的并联架构，正是对上述不足的针对性回应。

综合来看，现有国内研究在单一模型创新方面已有较多积累，但针对多模型集成精排管道的完整工程实现、前端交互平台与大数据推荐引擎的异构集成，以及特征工程阶段标签泄漏的系统性防控，仍缺乏系统性的公开论述，本文将就上述方面展开针对性研究与实践。

### 1.2.2 国外研究现状

在音乐推荐专项研究方向，Liu 等<sup>[11]</sup>提出情感驱动的个性化音乐推荐框架，通过对歌词与音频的情感分析构建用户情绪—偏好映射。Wan 等<sup>[12]</sup>设计了多视图增强图

注意力网络用于基于会话的音乐推荐，在多个公开数据集上取得了较好表现。Li 等<sup>[13]</sup>提出注意力自编码器用于内容感知型推荐，在交互稀疏场景下的性能优于纯协同过滤基线。Mao 等<sup>[14]</sup>提出轻量化音乐分类推荐网络，探索了分类任务与推荐任务联合建模的可行性。Paul 等<sup>[15]</sup>对协同过滤、内容过滤与混合推荐三类范式进行了冷启动与稀疏场景下的适用边界比较。Tran 等<sup>[19]</sup>将 Transformer 架构与认知科学中的记忆激活理论相结合，专门针对音乐收听场景中频繁出现的重复播放行为进行建模，为序列化音乐推荐中的用户行为分析提供了新的研究视角。上述研究在情感特征、内容建模与序列行为分析方面各有贡献，但多数工作聚焦于单一模型的改进，对多模型协同精排管道的完整工程集成探索相对不足。

在深度精排与行为序列建模方向，Chen 等<sup>[20]</sup>将 Transformer 架构引入推荐系统精排层，提出行为序列 BST，通过多头自注意力机制对用户历史行为序列中的时序依赖关系进行建模，相较于注意力池化类方法在点击率预测任务上展现出一定的性能优势。该工作表明 Transformer 在用户行为序列建模中具有较大潜力，为本文将行为序列 Transformer 引入音乐粗排场景提供了重要的方法论参考。

在大规模工程方向，Niu 等<sup>[16]</sup>基于分布式计算框架实现了大规模协同过滤音乐推荐，验证了分区策略对矩阵分解训练效率的提升作用。Johnson 等<sup>[17]</sup>提出的高性能向量近邻检索库已广泛应用于工业界推荐系统的召回层。Wang 等<sup>[18]</sup>通过神经因果图协同过滤对交互数据中的混淆因素进行显式建模，对缓解观察偏差对推荐结果的影响具有一定参考价值。上述工程方向的研究多聚焦于单一技术组件的优化，对多组件端到端集成方案的系统性论述相对有限。

综合来看，当前国内外研究多聚焦于单点模型与特定场景的算法突破。然而，在真实的音乐消费语境下，涵盖系统性特征防泄漏、多模型异构集成以及前后端深度集成的全链路工程实践，仍缺乏公开、可复现的文献支撑。本文将围绕上述工程痛点展开系统性构建，为中小型企业推荐系统提供的理论支持与参考价值。

### 1.3 论文研究内容

本文将重点解决以下四个核心问题：

（1）大规模音乐数据的高效治理与无泄漏特征工程

如何将基于 KKBOX 数据集构建全量 ETL 导入与特征工程管道，并采用 OOF（Out-of-Fold，折外预测）目标编码方案以消除训练集与验证集之间的标签泄漏风险，为粗排层与精排层各模型分别构建独立的多维特征集供模型训练使用。

（2）四层推荐管道构建与双模型 Stacking 集成

如何将 BST、DeepFM 和 LightGBM 共同置于四层推荐管道中，并实现三路召回通道汇聚候选歌曲，然后 BST 粗排将候选集压缩至合适数量，并使用 DeepFM 与 LightGBM 对粗选后的歌曲进行并行精排，通过 5 折交叉验证生成堆外预测，以逻辑回归元学习器学习两者的融合权重，形成两阶段 Stacking 集成，输出得分最高的歌曲，最终经 MMR 多样性重排得到所需的个性化推荐歌曲。

（3）MusicWeb 音乐平台全功能工程实现

如何基于 Java Web 技术栈构建 MVC 分层架构的在线音乐平台，覆盖用户认证与账户管理、个性化推荐展示、多源音乐检索与播放收藏、歌单管理及管理员后台等模块，通过事件驱动的异步上报机制采集用户播放完成率与反馈行为，经 C3PO 连接池高并发持久化写入，构建推荐反馈闭环。

（4）推荐系统离线评估体系构建

如何针对四层推荐管道与 Stacking 集成架构，设计合理的离线评估方案，在有效量化各管道模块实际增益的同时，横向比较各单体模型与集成框架的点击率判别能力，从而验证系统整体设计的合理性与有效性。

## 1.4 论文组织结构

本研究分为六章，附录提供特征概览与超参数配置等支撑材料。第一章梳理个性化音乐推荐的研究背景与国内外研究现状，确立本文的研究问题与工程实施策略。第二章对大数据处理、深度学习精排、集成学习与 Java Web 技术栈完成选型论证。第三章确立系统总体目标，从用户端、管理端与推荐引擎端三类参与者视角完成用例分

析，并明确性能与安全性需求。第四章对双模块总体架构、四层推荐算法流水线、前端交互及数据库展开详细设计。第五章按数据处理与特征工程、推荐算法实现、前端功能实现三条主线完成全链路工程落地。第六章通过消融实验与模型 AUC 对比实验两组离线评估，对各推荐模块的实际贡献进行量化验证。

## 2 系统开发技术与算法选型

基于第一章对国内外研究现状的梳理，深度学习模型与多模型融合技术是解决音乐推荐中用户行为稀疏、偏好动态变化等问题的有效途径。本系统将选用大数据处理技术、三路召回技术、BST、DeepFM 与 LightGBM 精排集成和 MMR 算法相结合的技术方案。本章将具体讲述这些技术为何选用于本系统，以及如何应用。

### 2.1 大数据处理与召回技术

PySpark 是由 Apache Spark 开发的 Python 接口，以内存计算为核心，支持分区裁剪与谓词下推等查询优化手段，适合对超大规模结构化数据进行分布式预处理。本系统选用 PySpark 完成 KKBOX 数据集的 ETL 工作，将 229 万首歌曲元数据与 737 万条用户行为记录批量导入 MySQL，有效解决了传统单机处理大数据集时的内存瓶颈。Redis 是基于内存的键值存储系统，具备亚毫秒级读写能力与自动过期回收机制。本系统将热门歌曲列表、新歌列表、用户收藏、播放历史、播放链接等高频查询结果存入 Redis，设置不同的分级过期策略，访问失败时自动降级至数据库直查，以保障高可用的同时有效降低数据库并发压力。

协同过滤召回采用 ALS 算法，将用户-歌曲隐式反馈矩阵分解为低维潜在因子空间，以置信度加权机制处理播放次数等隐式交互信号。相较于传统基于用户或物品的协同过滤，矩阵分解形式在数据稀疏场景下具有更强的泛化能力，且天然支持分布式并行训练，适合大规模数据的候选生成需求。向量召回采用 Facebook 开源的 FAISS 框架，基于离线预训练所得的歌曲嵌入向量构建余弦相似度精确近邻索引，在百万级候选库中快速定位与用户当前兴趣向量相近的歌曲，以弥补协同过滤在内容语义层面的不足。热度兜底策略作为第三路召回，用于保障新用户冷启动时的候选覆盖。三路召回合并去重后产生约 600 首候选，交由排序层进行后续处理。

### 2.2 核心推荐算法选型

BST 通过多头自注意力机制对用户历史播放序列建模，能够感知行为的时序位

置与序列内部依赖关系，捕捉比均值池化更精细的动态兴趣表征。音乐偏好具有明显的时序演变特征，近期播放行为对当前兴趣的反映强于久远记录，BST 在序列感知方面的优势使其适合承担对约 600 首候选的快速初步排序，将候选集压缩至合适规模以降低后续精排的计算压力。

精排阶段并行部署 DeepFM 与 LightGBM 两个模型。DeepFM 通过共享嵌入层将 FM 组件与 DNN 组件联合训练，在不依赖手工特征组合的前提下同时捕获低阶与高阶特征交叉，对推荐场景中大量稀疏类别特征具有较好的泛化能力，适合处理用户属性、歌曲属性与交互上下文多类型特征并存的复杂输入。LightGBM 基于梯度提升决策树框架，采用叶优先生长策略与直方图分箱算法，在高维稀疏特征下兼顾训练效率与泛化能力，能有效建模用户侧、歌曲侧与上下文侧 59 维混合特征中的非线性交叉关系。DeepFM 的隐式深度特征交叉与 LightGBM 的显式工程特征建模形成互补，两路评分共同送入集成层。

集成阶段采用 Stacking 框架，以 DeepFM 与 LightGBM 作为基学习器，以逻辑回归作为元学习器，通过时序交叉验证生成折外预测，在折外预测构成的元特征空间中自适应学习两个基学习器的最优融合权重，经约束优化后输出最终融合概率。重排阶段采用最大边际相关算法 (MMR) 对融合概率排序后的候选列表施加多样性软约束，以相关性与多样性惩罚的加权组合为评分标准，通过贪心迭代逐步选入与已选集合相似度最低、同时融合得分最高的歌曲，最终输出推荐结果。

## 2.3 Java Web 核心技术栈

MusicWeb 前端服务基于 Jakarta EE 规范开发，采用五层 MVC 架构，其中实体层定义用户、歌曲、歌单等核心数据模型；数据访问层封装各业务表的增删改查操作，实现存储逻辑与上层解耦；业务逻辑层处理推荐刷新等跨数据访问层的复合业务；控制层基于 Servlet 解析 HTTP 请求并调度业务逻辑；视图层基于 JSP 负责服务端数据渲染与页面返回。此外，数据库连接采用 C3P0 连接池统一管理，凭证通过运行时参数注入，避免硬编码。缓存层基于 Jedis 连接池访问 Redis，并在 Redis 不可用时自动

降级至数据库直查，保障系统高可用性。五层分离的设计降低了各模块间的耦合程度，为系统功能扩展提供了良好的架构基础。

## 2.4 本章小结

本章系统性梳理了推荐架构底层技术栈的选型逻辑与工程映射关系。分布式 ETL 预处理环节依托 PySpark 框架应对海量数据流转，针对热门曲目、用户收藏及播放历史等高并发访问场景，引入 Redis 实施分级缓存机制，以降低关系型数据库的读写负载。召回阶段采用 ALS 协同过滤、FAISS 向量检索与热度兜底三路并行策略，分别从用户行为相似性、内容语义相似性和流行度三个维度生成候选集，合并去重后输入粗排阶段。

排序管道遵循效率与精度分离的设计范式。粗排节点选用 BST 行为序列 Transformer，凭借其多头自注意力机制对用户近期播放序列快速评分，将候选集压缩至适合深度模型精排的规模。精排节点并行部署 DeepFM 与 LightGBM 两路模型，前者从稀疏特征的高阶静态交叉维度评估用户-歌曲匹配程度，后者以 59 维手工构建特征建模非线性交叉关系，两路互补信号均以 5 折 OOF 折外预测的形式送入 Stacking 集成框架。元学习器的自适应权重分配将两路信号融合调优，随后经重排层进行多样性软约束筛选，最终得到推荐结果。

在前端服务层面，MusicWeb 严格遵循 Jakarta EE 规范，以五层 MVC 架构实现各功能模块的职责分离，配合 C3P0 数据库连接池与 Jedis 缓存代理，系统构筑了应对高并发推荐请求的可用性基石。上述技术体系的确立框定了本研究的工程边界，为后续需求拆解及核心业务模块的代码落地提供了确切的技术指引。

### 3 系统需求分析

基于第二章对大数据处理技术与核心推荐算法选型的梳理，系统的技术路径已基本确立。在此基础上，需要进一步明确系统的业务功能边界与具体用户交互需求，为后续的架构设计与工程实现提供明确依据。本章将从系统总体目标、用例分析以及性能与安全需求三个方面，对系统需求进行全面分析。

#### 3.1 系统总体目标

本系统的设计目标是构建一套面向在线音乐场景的双模块个性化推荐平台，由 MusicWeb 用户交互模块与 MusicMode 推荐引擎模块协同构成。系统在支持音乐播放、收藏、搜索等基础业务功能的同时，以大数据技术为底层基础设施，以多模型集成精排管道为算法核心，以 Java Web 平台承载用户交互入口，并通过隐式反馈采集形成推荐优化的数据闭环。

如图 3-1 所示，系统从顶层划分为用户端、后台管理端与推荐引擎端三大功能模块。用户端承载账号管理、音乐检索与浏览、在线播放、收藏与歌单管理、个性化推荐展示以及播放历史查询六项核心业务。后台管理端覆盖数据概览、用户账号管理、申诉审核管理以及歌曲与歌单管理四项管理能力。推荐引擎端执行多路候选召回、多模型异构精排、多样性重排、隐式反馈采集以及定时推荐更新五项算法计算任务。三大模块通过共享 MySQL 数据库完成数据流转与状态同步。



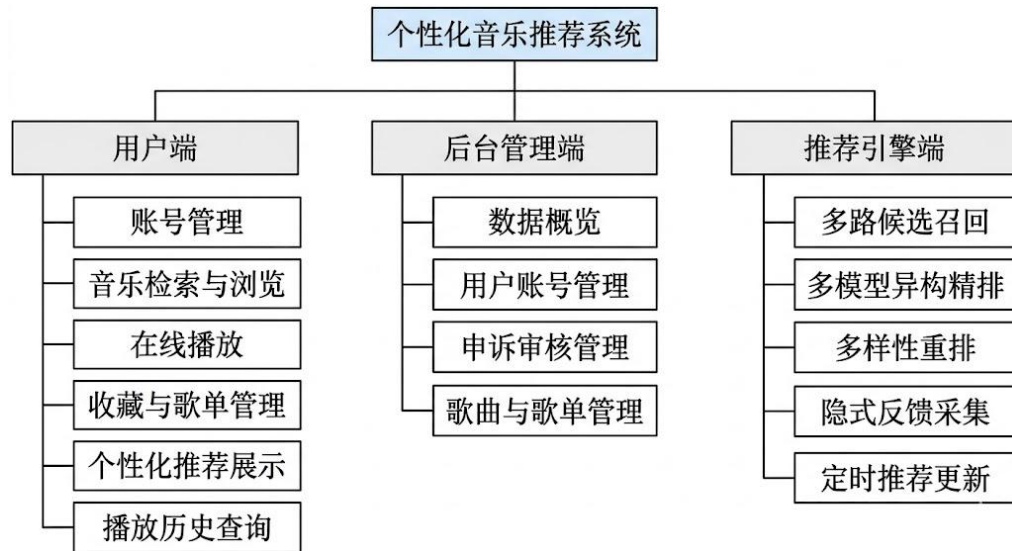


图3-1 系统总功能结构图

## 3.2 系统用例分析

### 3.2.1 参与者识别

系统涉及三类执行主体。普通用户是系统的主要交互者，涵盖音乐收听、歌单管理、推荐浏览及隐式反馈产生等全部前台操作链路。系统管理员以独立身份入口接入后台管理模块，负责用户账号审查、歌曲资源维护、申诉处置及平台数据统计等管理监控职能。推荐调度任务作为第三类隐性参与者，以定时批处理脚本的形式在后端自动触发全量推荐重算，无需人工干预即可完成候选召回、精排融合与推荐结果写库的完整执行链路。

### 3.2.2 核心用例图

本系统按照用例体系按执行主体拆分为用户端与管理端两个视图，分别如图 3-2 与图 3-3 所示。

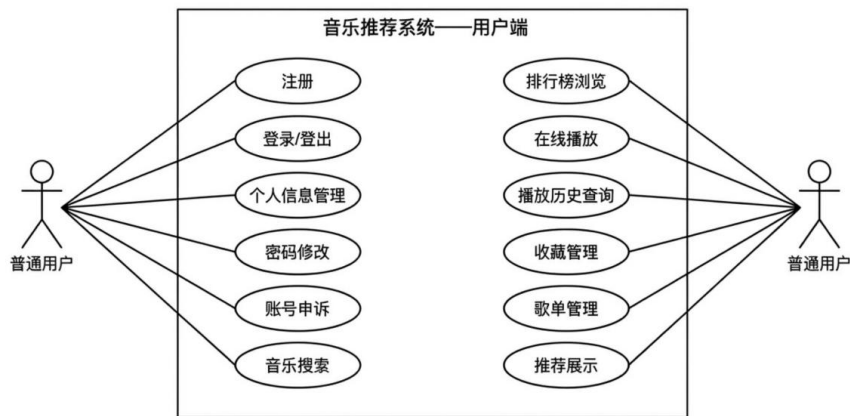


图3-2 用户端用例图

图 3-2 展示了普通用户在系统内的完整交互入口。注册用例完成账号基础信息录入后，系统引导用户填报流派与艺术家偏好，推荐引擎凭此生成冷启动候选集，使新用户首次登录即可获取个性化内容。推荐展示用例读取 MusicMode 引擎离线预计算的排序结果，用户在该界面触发在线播放后，前端计时器同步采集有效收听时长，离开页面时经异步请求将完播率上报至服务端写入反馈表。当完播率低于 20%的播放行为被标记为跳曲，触发对应内容的冷却降权处理。此外收藏管理用例中的收藏操作则作为强正向信号写入推荐反馈记录，上述行为信号构成推荐引擎下一轮迭代的标注数据来源。

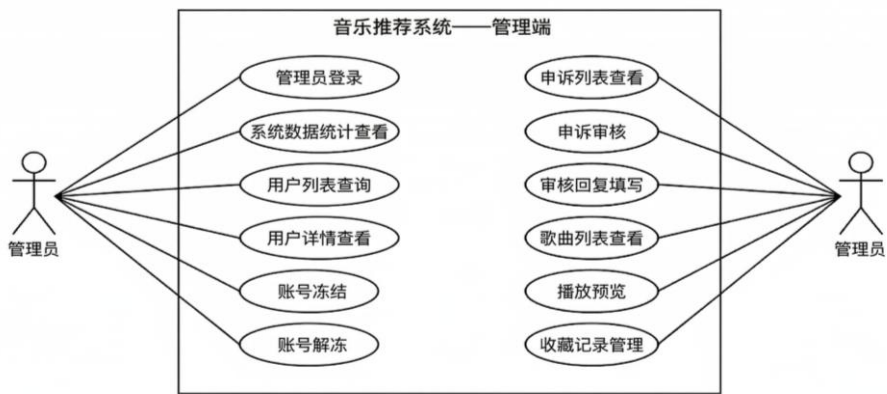


图3-3 管理员端用例图

图 3-3 展示了管理员在后台视图中的操作集合。账号冻结与账号解冻两个对称用

例联动账号申诉流程，管理员对违规账号执行冻结并记录截止时间后，被冻结用户可经前台申诉入口提交申请，申诉审核用例处置完成后，系统依据审核结果自动触发账号状态变更并向用户联系邮箱发送通知。歌曲列表查看与播放预览两个用例联动，管理员可在列表界面直接触发音频播放，验证入库歌曲资源的可用性。

### 3.2.3 推荐引擎端用例

音乐消费行为以隐式信号为主，用户鲜少主动评分，系统只能从播放时长、跳曲与收藏等间接行为中推断偏好。这种隐式性决定了单一召回策略难以同时覆盖行为丰富的活跃用户和无历史记录的新启动用户，因此推荐引擎须支持多路独立召回通道并行运作，分别从行为相似度、内容语义相似度与流行度三个维度产生候选，并在合并去重后形成统一的候选集。多路召回带来了较大规模的候选池，直接以高复杂度模型对全量候选评分的计算开销难以承受，系统因此须设计逐级压缩的多阶排序机制，先以具备序列行为感知能力的模型对全量候选完成初步评分压缩，再由具备更强特征交叉建模能力的多路深度精排模型对筛选后的子集实施精细评估。

此外，推荐分数最优的候选集若不加约束地直接输出，往往会因模型偏向热门风格而导致推荐结果的内容趋同，使用户陷入单一流派的信息茧房。为此，排序完成后须引入多样性约束处理，在相关性与列表风格丰富度之间取得平衡。并且用户偏好随收听深度持续演化，推荐引擎须具备采集播放完成率与收藏行为等反馈机制，对持续收到负向交互的内容设置冷却屏蔽，防止无效内容重复曝光。由于推荐计算开销较大，无法在用户访问时实时执行，系统须通过定时调度在每日固定时间窗口内完成全量用户的推荐更新，保证用户访问时能即时读取最新的推荐结果。

## 3.3 系统性能与安全性需求分析

### 3.3.1 性能需求

客户端界面须将服务端数据返回后的交互响应延迟控制在可接受范围内，保障用户操作的流畅性。针对音乐搜索、排行榜检索等高频只读查询场景，Redis 缓存层在

数据写入时同步完成预加载，请求命中缓存后绕开数据库直查路径，有效降低关系型数据库在并发高峰期的读负载压力。此外，推荐引擎须在预设的定时窗口内完成面向全量注册用户的候选召回、多模型排序评分与推荐结果写库操作，保证每日推荐更新的计算吞吐能力。系统须通过 C3P0 数据库连接池统一管理并发数据库连接，在高并发请求峰值期间弹性扩展可用连接数，保证请求不因连接资源耗尽而长时间阻塞，访问低谷期则自动回收空闲连接，降低数据库端的持续资源占用。

### 3.3.2 安全性需求

数据库操作须采用参数化查询方式将用户输入数据与 SQL 语句逻辑分离，降低 SQL 注入风险，前端表单同时集成格式校验与长度约束机制，保证提交数据的合法性。系统在将用户内容回显至页面时，须对特殊字符进行转义处理，防止恶意脚本被注入至其他用户的浏览器上下文，保护用户会话数据的安全性。在身份认证层面，系统以服务端会话机制维护用户登录状态，所有受权限控制的页面与接口均须在服务端完成会话有效性验证，拦截未授权访问请求。管理员账户经独立身份认证后方可进入后台模块，普通用户无法访问任何管理端接口，服务端对两类角色实施严格的权限映射，确保接口调用与业务操作的合法性边界清晰。此外，系统须对被冻结或注销账号的登录请求实施拦截，并向用户展示明确的账号状态说明与申诉入口，防止异常账号继续访问受保护资源，当管理员执行的冻结与解冻操作时须即时生效。

## 3.4 本章小结

本章围绕系统需求分析展开论述，为后续架构设计提供了明确的功能边界与设计方向。总体目标层面，系统定位于 MusicWeb 与 MusicMode 双模块协同构成的个性化音乐推荐平台，并以系统总功能结构图直观呈现了用户端、后台管理端与推荐引擎端三大模块的职能划分。用例分析层面，制定出普通用户、系统管理员与推荐调度任务三类参与者，分别从用户交互、后台管理以及算法计算三个维度拆解各自承载的用例职能，并以用例图展示参与者与系统功能之间的关联关系。而推荐引擎端的用例描述着重阐明了多路召回、多阶排序、多样性后处理与隐式反馈采集各项功能的设计动

因。性能与安全需求层面，对交互响应延迟、推荐计算吞吐、数据库并发连接等性能指标以及注入防护、身份认证与权限隔离等安全约束进行了明确规定。上述功能与需求的明确为第四章系统架构设计提供了框架支撑。

## 4 系统设计

基于第三章确立的系统功能用例边界与性能安全约束，系统的需求范围已明确界定。设计阶段的任务是将上述需求转化为可落地的技术架构方案，为后续的工程实现提供明确依据。本章将依次围绕系统总体架构、前端交互设计、推荐算法流水线与数据库结构四个方面展开具体设计。

### 4.1 系统总体架构设计

本系统采用双模块解耦架构，由面向用户交互的 MusicWeb 模块与面向算法计算的 MusicMode 模块协同构成，两者以 MySQL 关系型数据库为共享数据层进行集成，通过异步批处理机制实现数据闭环。如图 4-1 所示，系统整体架构由左侧 MusicWeb、右侧 MusicMode 及底部共享数据层三部分组成：MusicWeb 持续将用户行为信号写入共享数据库；MusicMode 周期性从数据库读取行为数据完成推荐计算后将结果回写；MusicWeb 再从数据库读取推荐结果渲染至用户界面，形成完整的数据驱动推荐闭环。

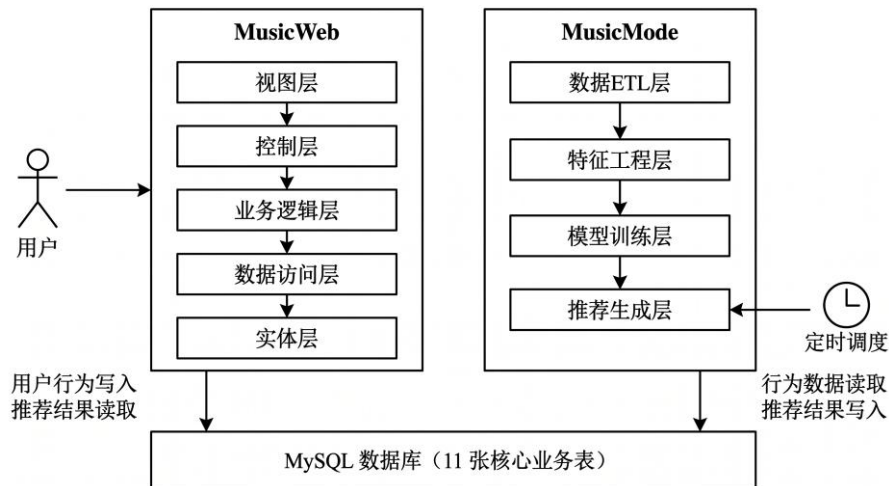


图4-1 MusicWeb + MusicMode双模块整体架构图

#### 4.1.1 MusicWeb 前端模块架构

MusicWeb 是系统的用户交互平台，使用 Java 和 Jakarta Servlet/JSP 技术栈来建立，其架构为 MVC 五层架构，由视图层、控制层、业务逻辑层、数据访问层、实体

层五部分组成，各个部分之间职责分明，层次之间互相依赖关系只有一层，即下层向上传递。

#### （1）视图层

视图层由 JSP 动态页面、JavaScript 行为脚本、CSS 样式表三部分组成，用来向用户浏览器输出可以交互的 HTML 界面。前端层利用异步请求与控制层进行交互，可以实现推荐列表刷新、收藏状态变更等无刷新的动态操作，从而减小由于整页加载而给用户造成的感知延迟。

#### （2）控制层

控制层是由一组根据业务功能独立出来的 Servlet 控制器所组成，它集中将表现层发出的 HTTP 请求传送到下一层（JSP 层）上去执行。根据业务域来划分控制器集合，共包含用户认证、个人信息管理、音乐服务、行为上报、内容管理、推荐服务、管理端这 7 个功能域。控制层不会直接去处理数据库的操作，只会对参数进行解析，对权限进行校验，并且会安排流程的调度工作，从而保证自身轻量化。

#### （3）业务逻辑层

业务逻辑层负责承载跨 DAO 的复合操作与推荐服务的协调逻辑。推荐服务组件负责从数据库读取由推荐引擎预计算的推荐结果，并结合用户当前的收藏状态与历史播放记录对推荐列表进行过滤与补全，最终向控制层返回可直接渲染的推荐歌曲列表。

#### （4）数据访问层

数据访问层用 C3P0 连接池来获得数据库连接，对系统全部 11 张核心业务表的读写操作做统一封装，将 SQL 语句的细节和结果集的映射逻辑都隐藏起来。各个数据访问组件按照业务实体分成了用户信息、歌曲元数据、播放历史、收藏关系、歌单数据、推荐记录和反馈数据这些主要实体的持久化工作，全部 SQL 操作都采取了参数化查询的办法来规避 SQL 注入这种安全问题。此外，数据访问层还通过 RedisUtil 组件对热点数据提供缓存读写接口，降低数据库在高频读场景下的负载压力。

#### （5）实体层

实体层由各核心业务实体类构成，涵盖用户、歌曲、歌单、收藏关系、播放历史

与申诉等业务对象，与数据库各核心业务表一一对应。各实体类仅封装字段属性与访问器方法，不含业务逻辑，在控制层、业务逻辑层与数据访问层之间作为统一的数据载体流转，避免各层直接依赖松散的键值结构，从而降低层间耦合程度。

#### 4.1.2 MusicMode 推荐引擎模块架构

MusicMode 为系统的算法计算部分，使用 Python 和 PyTorch 深度学习库构建，采用定时批处理的方式运行。模块内部自上而下分为数据 ETL 层、特征工程层、模型训练层和推荐生成层四个功能层次，各层次的输出物通过磁盘序列化文件或数据库表实现跨层传递，支持各层次独立执行与增量更新

##### （1）数据 ETL 层

ETL 层负责从 MySQL 数据库中抽取原始歌曲与用户行为数据，执行数据清洗、格式规范化与样本构造。系统使用 PySpark 分布式计算框架通过 JDBC 接口读取全量数据，并按歌曲唯一标识进行增量去重，防止重复写入。训练样本以用户 30 天内重复收听行为作为正样本标准，以等比例随机负采样构建负样本，输出结构化训练样本集供特征工程层使用。

##### （2）特征工程层

特征工程层对 ETL 层输出的原始样本进行特征提取，为三个模型分别构建独立的特征集，涵盖用户行为统计、歌曲属性、交互匹配与序列行为等维度。所有类别型特征均采用 OOF 折外目标编码，在 K 折交叉验证框架内每折的编码值从其余 K-1 折数据统计得出，有效规避目标变量泄露。

##### （3）模型训练层

模型训练层以 BST 粗排模型进行独立训练，DeepFM 与 LightGBM 作为精排层的两个基模型，分别使用各自的特征集完成 GPU 加速训练后，输入到精排集成阶段，使用两个基模型在验证集上的折外预测概率作为元特征，训练逻辑回归元学习器进行融合，各模型权重文件序列化存储后供推荐生成层直接加载。

##### （4）推荐生成层



推荐生成层完成四层推荐管道的完整执行逻辑，按三通道候选召回、BST 粗排筛选、DeepFM 与 LightGBM 双模型集成精排、多样性重排的顺序依次处理，最终将每位用户的个性化推荐列表批量写入数据库。

### 4.1.3 双模块交互与部署设计

MusicWeb 与 MusicMode 两个模块之间采用共享的 MySQL 数据库作为唯一数据交换媒介，建立了异步松耦合集成架构。数据库在此架构中承担持久化数据转储的职能，即使推荐引擎在批处理计算中途异常中断，前端依然能读取已存库的推荐结果，保证推荐服务不因算法端故障而中断。数据表即交换媒介的设计，也便于通过直接检索数据表观测跨模块的信息流转状态，降低排查难度。

在系统部署层面，MusicWeb 被构建为标准 war 包运行于 Apache Tomcat 容器中，作为常驻应用提供实时交互访问。MusicMode 算法引擎以 Python 脚本群形态存在，由操作系统的定时任务按时触发全量推荐重算。MySQL 与 Redis 均配置为独立的守护进程，系统外围配置了专用的批处理调度脚本统筹各组件的启动依赖关系，顺序拉起各服务，确保前端在对外接收请求前所需的前置组件均已就绪。

## 4.2 前端交互设计

### 4.2.1 页面总体结构设计

系统页面整体采用统一的头部导航布局。顶部导航栏在未登录状态下仅展示首页和登录引导文字，已登录状态下扩展为首页、发现、排行榜、我的收藏四个链接，并在右侧显示用户头像、用户名、退出与设置入口。各页面样式由独立的 CSS 文件管理，主内容区依照当前功能模块动态渲染对应视图，实现样式隔离与可维护性。

系统页面分为两类：首页为公开访问页面，展示宣传内容与新歌推荐；已登录用户访问时将展示用户主页、歌曲搜索、歌单详情、个人设置等功能页面，各页面在服务端通过 Session 检查登录状态，未登录用户访问受保护页面时将自动跳转至登录入口。系统整体页面导航层次如图 4-2 所示。

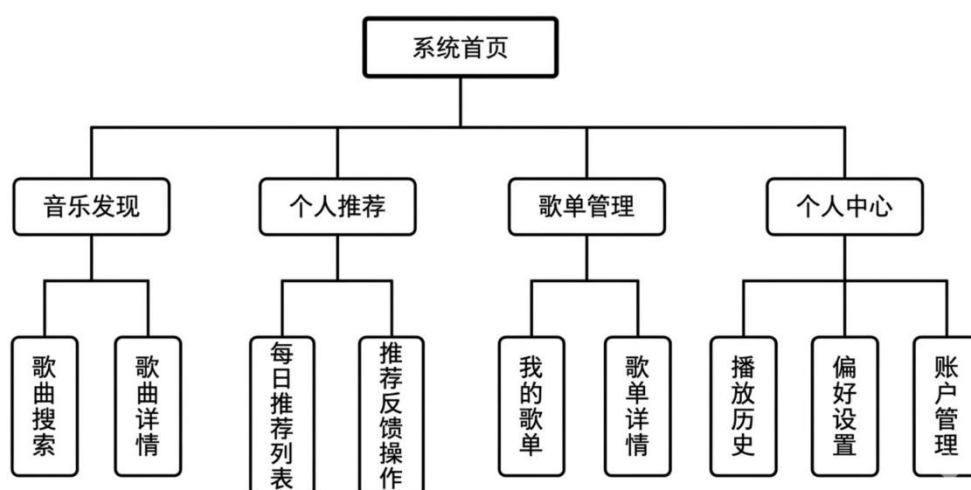


图4-2 系统页面导航层次结构图

图以树形层次呈现了系统各功能页面的组织关系。一级节点为系统首页，二级节点为四大功能模块，三级节点为各模块下的具体页面。用户可通过顶部导航栏直达各二级模块，再经页面内部交互跳转至对应三级页面，交互层级不超过三级，降低操作的认知负担。

#### 4.2.2 核心功能页面设计

系统核心功能页面共分为四个模块，各模块在交互设计上均以简洁高效为原则，以最少的操作步骤完成目标任务。音乐搜索模块以搜索框为唯一入口，配合多来源切换按钮支持跨平台检索，兼容搜索历史留存，搜索结果以列表形式逐条呈现，每条结果提供来源标签标注与播放、收藏操作入口。个性化推荐模块以有序列表展示当天 50 首推荐歌曲，播放、收藏与反馈三类交互操作均通过异步请求实时写入推荐反馈表，构成下一轮推荐迭代的数据输入。

歌单管理模块支持用户创建、编辑与删除自定义歌单，歌单详情页以分页列表呈现歌单内歌曲，支持按添加时间、歌手、专辑等维度排序，系统默认歌单仅支持查看与播放，不开放编辑与删除权限。个人中心模块以标签页形式展示个人信息管理、账户安全与账户管理三项功能，而播放历史则以独立页面呈现，以及内容屏蔽功能支持按流派与歌手设置屏蔽规则。

### 4.2.3 推荐交互与行为反馈设计

推荐交互设计的核心目标是快速采集用户行为信号并形成闭环反馈，如图 4-3 所示。推荐引擎离线完成候选生成与排序后将结果写入数据库，前端在用户访问时读取并展示当日推荐列表。用户在推荐页面触发的交互分为三类：播放行为触发前端计时器实时记录播放时长，用户离开时通过异步请求将完播率上报至后端；收藏行为作为强正向信号立即写入反馈记录；反馈不喜欢的行为被标记为负向交互，触发对应内容的降权处理。三类信号统一存入推荐反馈表，作为推荐引擎下一轮迭代的训练数据来源。

此外，用户可在个人中心主动声明对特定流派或歌手的屏蔽，系统采用递增冷却机制，每次屏蔽操作令屏蔽计数加一，冷却到期日随计数增大而延长，到期后屏蔽状态自动失效，防止单次操作造成永久性内容排除。上述多路行为信号共同构成推荐系统的在线反馈闭环。

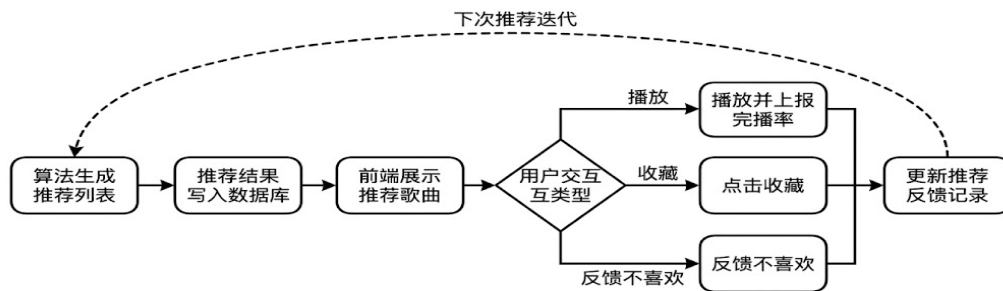


图4-3 推荐交互行为反馈闭环流程图

### 4.3 推荐算法流水线设计

推荐算法流水线是本系统实现个性化推荐的核心计算主干，采用四层串行结构，如图 4-4 所示。召回层在全量歌库中并行生成约 600 首候选，粗排层以具备序列行为感知能力的 BST 模型对全量候选快速评分压缩，精排集成层由 DeepFM 与 LightGBM 两路异构模型评分后经元学习器融合，重排层在最终输出前施加多样性约束，避免推荐列表内容同质化。四层管道各司其职，相互解耦，支持各层次独立迭代与替换。

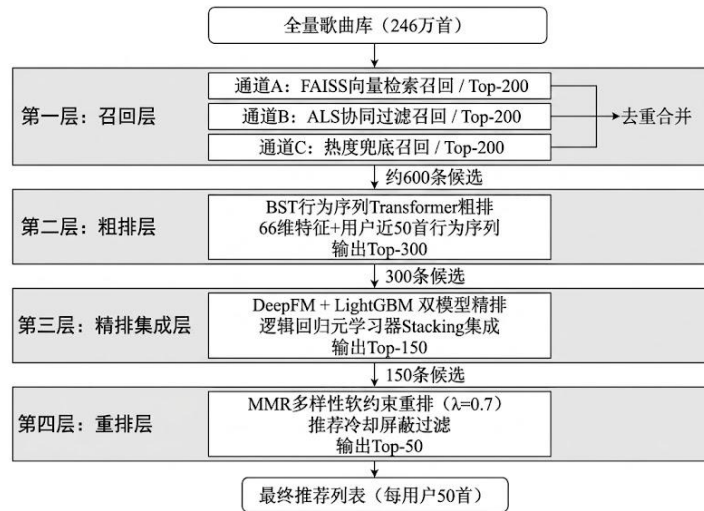


图4-4 四层推荐管道总体流程图

### 4.3.1 召回层设计

召回层的核心设计目标是在计算效率约束下实现对高质量候选歌曲的高覆盖率捕获。系统设计了三条相互独立、特性互补的召回通道，并行执行后去重合并构建统一候选集。如图 4-5 所示，三通道分别基于向量近邻检索、热度排序与协同过滤三种不同的信息源进行候选生成，以覆盖不同类型的用户兴趣信号。

#### 通道 A：FAISS 向量检索召回

根据用户的历史行为构建用户兴趣向量，使用 FAISS 内积索引在全量歌曲向量空间中检索内积相似度最高的前 200 首候选。向量索引在离线阶段完成预计算与持久化，在线召回阶段仅需执行向量查询，满足大规模推荐场景下的吞吐量要求。

#### 通道 B：热度兜底召回

热度兜底通道以歌曲总播放量与用户偏好艺术家加权分的组合形式排序，补充前 200 首候选。该通道主要面向行为信号极度稀疏的新用户场景，当向量检索与协同过滤通道无法生成有效候选时，依靠高热内容保证推荐列表的基本可用性。

#### 通道 C：ALS 协同过滤召回

协同过滤通道基于 ALS 隐式反馈矩阵分解捕捉用户间的协同信号，向每位用户提供 200 首候选。针对用户行为数据的稀疏性差异，系统按活跃程度设计三级降级路

由：高活跃用户直接使用 ALS 隐因子向量排序召回；中等活跃用户改用多维内容相似度加权召回；冷启动用户以注册时填写的流派与艺术家偏好作为兜底召回依据。

三个通道各自生成候选列表后，系统以歌曲唯一标识为键进行集合去重，同时剔除用户近期已播放及处于推荐冷却期的歌曲，合并得到约 600 首不重复的候选集，传入下游精排层。

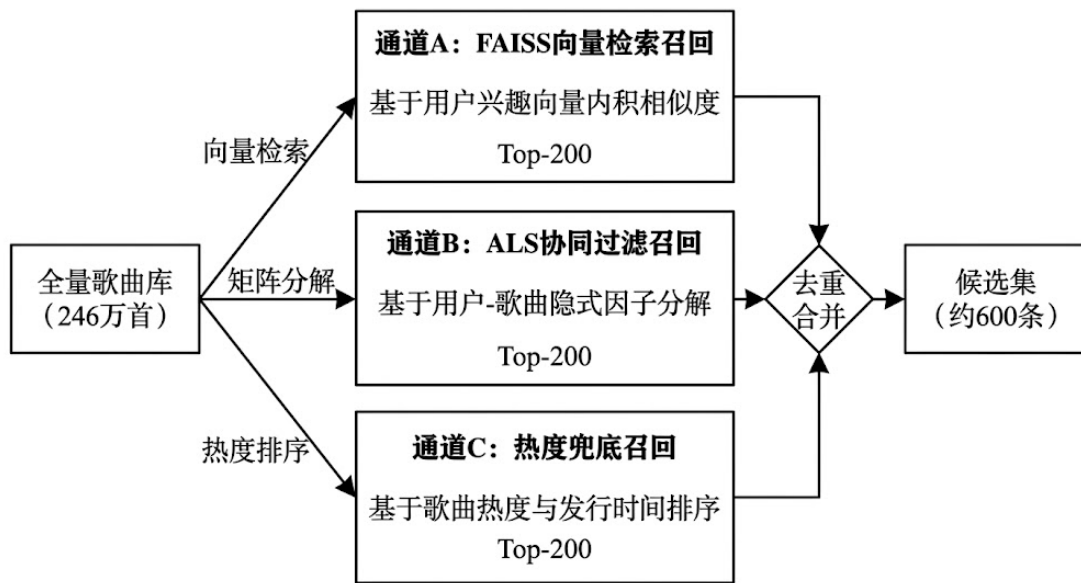


图4-5 三通道并行召回结构图

### 4.3.2 排序层设计

候选集完成去重合并后，经排序层逐级压缩以提升候选精度。排序层分为粗排、精排与集成三个阶段，以效率优先到精度优先的递进策略逐步缩减候选规模。

粗排阶段以 BST 行为序列 Transformer 为核心，对约 600 首候选进行批量评分。通过对用户近期播放序列实施时序编码和多头自注意力机制感知历史播放行为的时序依赖关系，对每首候选输出序列感知评分，按评分降序截取前 300 首递送精排阶段。

精排阶段由 DeepFM 与 LightGBM 两个模型共同承担，从静态特征交叉与显式工程特征建模两个互补维度对前 300 首候选独立评分。DeepFM 以静态用户-歌曲特征为输入，建模稀疏特征间的高阶交叉关系后输出评分。LightGBM 以 59 维混合特征

为输入，凭借梯度提升树对行为统计、热度衰减与流派偏好等聚合特征的建模能力给出独立评分。两路模型的独立概率预测值共同传入集成阶段。

集成阶段通过 Stacking 元学习器对 DeepFM 与 LightGBM 的评分进行融合。离线训练阶段，两个基模型基于全局播放时间戳进行 5 折时序交叉验证，生成各自的折外预测概率，将两组折外概率向量拼接为元特征矩阵，在验证集真实标签的监督下训练逻辑回归元学习器，并将权重参数序列化存储。推荐生成时，元学习器对前 300 首候选的两路评分进行融合，按集成得分降序截取前 150 首传递至重排层。

### 4.3.3 重排层设计

重排层在精排集成层输出的前 150 首候选上，通过 MMR 算法实施多样性重排并过滤冷却期候选，将最终推荐规模压缩至每位用户 50 首。集成评分以单曲维度的播放概率为优化目标，不对推荐列表的整体内容分布加以约束，仅依赖集成概率排序往往导致推荐列表风格集中。重排层采用 MMR（最大边际相关，Maximal Marginal Relevance）算法，在候选相关性与已选列表多样性之间进行权衡，其得分公式见式(4-1)所示：

$$\text{MMR}(d) = \lambda \cdot \text{Rel}(d) - (1 - \lambda) \cdot \max_{d_j \in S} \text{Sim}(d, d_j) \quad (4-1)$$

式中  $d$ ——当前待评估的候选歌曲；

$\text{Rel}(d)$ ——该候选歌曲的归一化集成评分，取值范围为[0,1]；

$\text{Sim}(d, d_j)$ ——候选歌曲  $d$  与已选列表中歌曲  $d_j$  基于 FAISS 嵌入向量计算的余弦相似度；

$S$ ——已选列表，即当前已通过贪心选取加入推荐结果的歌曲集合；

$\lambda$ ——相关性权重系数，取  $\lambda=0.7$ ，即相关性占 70%，多样性惩罚占 30%。

算法按 MMR 得分贪心地逐一筛选候选，每轮选取当前 MMR 值最高的歌曲加入已选列表，直至选满 50 首。系统对同一艺术家的入选数量设有上限，超出上限的候选不做直接剔除，而是对其 MMR 得分施加惩罚系数后重新参与排序，在保障多样性的同时避免因强制过滤损失高相关性候选。

重排层在贪心筛选过程中同步执行冷却过滤，实时剔除处于推荐冷却期内的候选歌曲，防止收到负向反馈的内容再次曝光。经多样性重排与冷却过滤后，最终为每位用户生成 50 首个性化推荐结果并批量写入数据库。

## 4.4 数据库设计

### 4.4.1 数据库结构设计

系统采用 E-R 模型对业务数据进行概念层抽象，共设计 11 张核心业务表，分别为用户表、歌曲表、播放历史表、用户歌单表、歌单歌曲关联表、推荐结果表、推荐反馈表、用户满意度反馈表、申诉表、用户屏蔽内容表和歌曲滚动统计表。

各表间的核心关联关系如下：用户与歌曲之间是多对多关系，每条播放行为以独立记录形式存入播放历史表；用户与歌单之间是一对多关系，歌单与歌曲之间通过歌单歌曲关联表建立多对多映射；推荐结果表与用户之间是一对多关系，推荐反馈表挂载于推荐结果表上，记录用户对各推荐歌曲的实际交互行为；用户满意度反馈表、申诉表与用户屏蔽内容表均以用户为中心，分别记录整体评价、申诉请求与内容屏蔽规则。歌曲滚动统计表与歌曲主表一对一关联，以独立表形式预聚合近 7 天与 30 天的播放量及热度指标，避免高频写入影响歌曲主表的读取性能。系统主要实体间的关系如图 4-6 所示。

概念模型中的 11 个实体依次对应 MySQL 数据库中 11 张业务表，通过外键约束与联合索引保证数据一致性与查询效率，各表的详细字段定义参见附表 1。

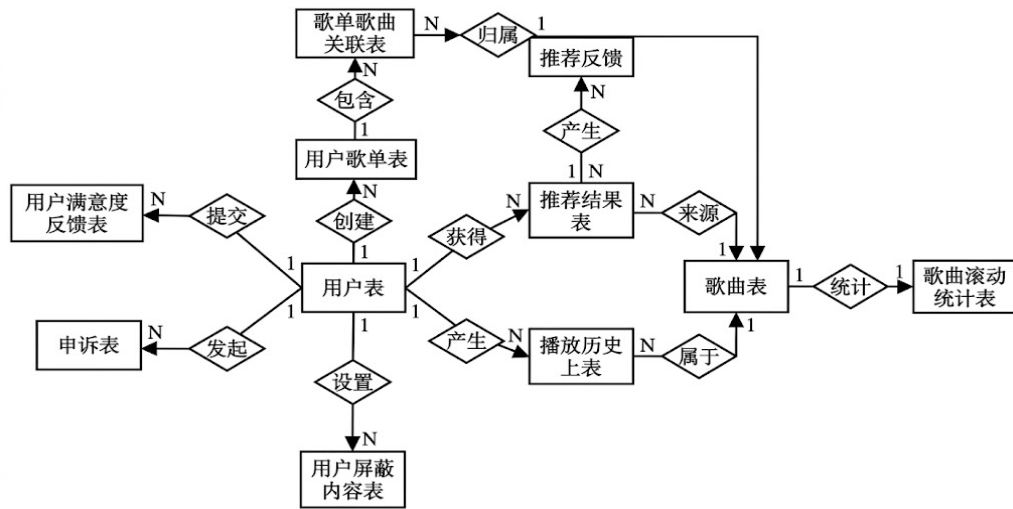


图 4-6 系统核心实体关系图

## 4.5 本章小结

本章在需求分析的基础上，详细阐述了系统的技术架构与核心模块设计方案。总体设计确立了 MusicWeb 与 MusicMode 双模块解耦的总体架构，明确了各模块内部的分层结构及基于共享数据库的异步交互方式。在此架构下，完成了前端交互设计，规划了各级页面导航与四大核心功能模块的交互策略，并建立了完整的行为反馈闭环。在推荐核心网络维度，设计了包含多路召回、BST 粗排、DeepFM 与 LightGBM 集成精排及 MMR 重排的四层算法流水线，详细规定了各层的候选规模演变与数据流转逻辑。最后，基于 E-R 模型完成了数据库结构设计，规划了对应业务需求的 11 张核心数据表。本章的系统设计方案实现了前后端与业务算法的有效解耦，为下一章的代码落地与系统实现提供了明确的蓝图指导。



## 5 系统实现

本章根据第四章系统设计方案，对各个核心模块的实现过程进行了详细的说明。全章按照开发环境配置、数据处理和特征工程、推荐管道算法实现、前端功能实现这四个主线来组织，结合工程实践中遇到的技术关键点以及调优细节，实现系统从数据处理到推荐生成的全流程。

### 5.1 开发环境与工具配置

本系统的开发与训练均在本地单机环境下完成，Python 端承担算法计算与模型训练任务，Java 端承担 Web 服务与页面渲染任务，两端共享同一 MySQL 实例实现数据联动。系统软硬件环境配置见附表 2 的表 5-1。

系统用 C3P0 连接池统一管理数据库连接，启动时预设 5 个连接，在并发峰值时最多可以增加到 20 个，空闲时间大于 300 秒的连接则会自动回收，防止 MySQL 服务端因为长期闲置而引起的连接积压导致超时断开。每次签出连接之前先做存活检查，配合 30 秒的定期心跳检测，保证连接池里的连接一直处在可用状态，最后所有的数据访问方法在异常处理完后归还连接，以确保出现异常不会造成连接泄露。具体的参数配置见附表 2 的表 5-2。

### 5.2 数据处理与特征工程实现

#### 5.2.1 数据介绍与 ETL 处理

本系统模型训练所用的核心数据来自 KKBOX 公开音乐数据集，是 KDD Cup 推荐系统竞赛数据，包含用户行为交互数据、歌曲元数据和用户画像数据三种结构化数据，一共包含了约 229 万首歌曲、约 737 万条历史收听行为记录和约 3.4 万名注册用户。除此之外，从网易云音乐端获取约 2 万首歌曲元数据，经清洗去重后合并入歌曲表，扩充了推荐候选池的覆盖范围。行为交互数据中目标变量是正负样本划分的依据，也是所有推荐模型训练与离线评价的基础监督信号，各个字段含义见附表 3 的表 5-3。

歌曲元数据由歌曲基础信息与歌曲扩展信息两部分合并构成，为系统构建歌曲侧内容特征提供原始数据支撑。其国际标准录音制品编码（ISRC）内嵌了发行国家与发行年份信息，经正则解析可提取歌曲来源地区与时效性特征，是特征工程阶段歌曲侧特征扩展的重要数据来源，各字段含义见附表 3 的表 5-4。

用户画像数据记录了 KKBOX 平台注册用户的基本属性信息，共 3.4 万名用户，用于构建用户侧静态属性特征。其中年龄字段存在大量异常值（如取值为 0 或明显超出合理范围），各字段含义见附表 3 的表 5-5。

ETL 处理阶段通过 PySpark 分布式计算框架，经标准数据库接口对歌曲信息、播放历史、用户信息、歌单关系这四张业务表进行全量抽取。在执行计划层，框架根据特征工程字段依赖关系来自动产生并应用列裁剪和谓词下推这两种优化策略。其中列裁剪只向数据源请求数据所需要字段，删除与建模无关的冗余字段，而谓词下推把过滤条件提前到数据源端执行，在磁盘读取阶段就丢弃不符合条件的行，而不是在内存里过滤掉。两项优化合计将磁盘读取传输量削减约 60%，有效降低了后续特征变换阶段的集群内存峰值压力。

### 5.2.2 数据清洗

原始数据有三类质量问题，需要在特征构建之前一一解决：

#### （1）缺失值填充

歌曲元数据中 `artist_name`、`genre_ids` 等字段有部分为空值，用语义占位符来代替其中等类型字段，数值型字段（`song_length`）用全局中位数来填补，保留数据行数，不带来极端偏差。训练行为数据里包含少量缺失行，直接舍弃来保证样本标签的完整性。此外对于异常值，则在分桶离散化阶段通过异常捕获机制将不可解析值映射为 "unknown" 类别，数值越界的异常值则自然归入最近的分桶区间，从而在不损失样本量的前提下规避异常值对模型的干扰。

#### （2）冷启动过滤

交互次数少于三次的用户由于行为信号非常稀疏，不能对统计特征进行可靠的估

计，将这些用户从训练集中剔除。保留至少被播放过一次的所有歌曲，使 Embedding 层能够有足够多的样本来学习，防止因为过度过滤而造成召回层候选集太小。

### （3）样本分布校验

经对 KKBOX 原始训练数据统计，正样本(target=1)约占 50.35%，负样本(target=0)约占 49.65%，正负样本天然接近均衡，无需执行额外的下采样操作。清洗阶段只用全量的有效样本参与模型的训练过程，避免由于人为采样造成的分布偏差。

## 5.2.3 特征工程

系统为粗排层与精排层各模型分别构建了独立的特征集，BST 粗排特征集共 66 维，DeepFM 精排特征集共 66 维，LightGBM 精排特征集共 59 维。三个特征集在稀疏特征种类与稠密特征组合上各有侧重，但均从用户画像、歌曲属性、行为统计、交叉匹配、时序动态和隐向量嵌入六个维度对"用户-歌曲"交互关系进行刻画，完整特征集见附表 3 的表 5-6。

### （1）用户侧特征

用户侧特征分为画像类和行为统计类两种。画像类特征有性别、城市、年龄分段、账户成熟度分段等，经分桶离散化后得到稀疏特征输入嵌入层。行为统计类特征是在全量历史播放记录基础上预计算出来的，包括用户累计播放量、历史平均完播率、流派偏好分布等。

### （2）歌曲侧特征

歌曲侧特征同样分为属性类与统计类两类。属性类特征包括流派标识、发行语言、演唱艺术家等。统计类特征包括全局累计播放量、全局平均完播率、归一化热度得分等。此外引入热度趋势指标以捕捉近期播放动态，其计算方式见式 5-1 所示：

$$\text{trend\_ratio} = \frac{c_{7d}+1}{c_{30d}/30+1} \quad (5-1)$$

式中  $c_{7d}$ ——表示该歌曲最近 7 天的播放次数；

$c_{30d}$ ——表示最近 30 天的播放次数。

两者均采用严格的左闭时间窗口计算，以确保特征计算的时间无泄漏性。该指标

大于 1 时说明歌曲近期热度加速上升，小于 1 时代表热度趋于衰退。

### （3）用户-歌曲交叉特征

系统对流派、艺术家、语言、来源国家四个方面的匹配度进行计算，统计出在用户历史播放记录中，该属性取值相同于候选歌曲的记录所占比例，记为用户对该属性属性匹配度。另外建立时段匹配特征，即当前时段是否在用户历史最高的三个收听时段之内，活跃日匹配特征，即当前日期是否在用户历史上最活跃的前三天之内，从时序上进一步对用户的习惯进行刻画。

### （4）时序行为特征

时序行为特征分成记忆衰减特征和滚动窗口统计特征两大类。记忆衰减特性用距离用户上次收听同一种歌曲天数表示（第-1 表示第一次听到这首歌），此时正在播放的这首歌总的收听次数。滚动窗口统计特征用严格的左闭窗口来计算，在 7 天和 30 天这两个时间粒度上对用户播放总量、用户近期平均完播率、歌曲播放总量进行统计，并且将热度趋势指标加入进来体现用户以及歌曲最近的情况。

### （5）SVD 隐向量嵌入特征

为将用户-歌曲协同交互信息压缩为低维稠密表示，系统基于截断奇异值分解对用户-歌曲交互矩阵执行矩阵分解，公式见式 5-2 所示：

$$R \approx UV^T \quad (5-2)$$

式中  $R$ ——为用户-歌曲交互矩阵（发生播放记为 1，否则为 0）；

$U$ ——为用户隐向量矩阵；

$\Sigma$ ——为奇异值对角矩阵；

$V^T$  为歌曲隐向量矩阵。

分解后分别取前 10 个奇异分量得到 10 维用户嵌入向量与 10 维歌曲嵌入向量，并对用户-艺术家交互矩阵执行相同分解得到 5 维嵌入向量，以及用户向量与歌曲向量的点积作为协同过滤评分，上述共 26 维嵌入特征作为稠密特征拼接至统计特征中。

### （6）OOF 目标编码

高基数稀疏特征的类别数目在数万到几十万之间，直接用标签编码的话就会出现

目标泄漏的风险，造成训练集评价结果过高。系统采用留出折叠目标编码（Out-of-Fold Target Encoding, OOF-TE）方案，其核心公式见式 5-3 所示：

$$TE_k(x) = \frac{C_{-k(x)} + \alpha \cdot \mu}{N_{-k(x)} + \alpha} \quad (5-3)$$

式中  $k$ ——为当前验证折编号；

$C_{-k(x)}$ ——为除第  $k$  折外的其余 4 折中特征取值等于  $x$  的正类样本数；

$N_{-k(x)}$ ——为对应的样本总数；

$M$ ——为全局正例率；

$A$ ——为平滑系数（本系统取  $\alpha=10$ ）。

OOF-TE 将训练集划分为 5 折，每一折样本的编码值仅由其余 4 折的统计量计算得出，每条样本的目标编码值均来自其未曾参与训练的数据，从目标上消除了特征计算环节引入的数据泄漏。推荐生成阶段则使用全量训练集统计量计算目标编码，以充分利用所有可用的历史信息。

#### 5.2.4 数据集时序切分

推荐场景下训练集与验证集的划分必须严格按照时间顺序进行，不能采用随机分组方式，否则验证集中会混入时间上早于训练集的样本，导致模型“提前看到未来数据”，使离线 AUC 虚高而失去参考意义。本系统对不同模型采用对应的时序切分策略：BST 粗排层与 DeepFM 精排层均采用全局时序切分，将全体样本按全局播放时间戳升序排列，统一取末尾 10% 作为验证集。DeepFM 前 90% 区间另切分 5 折生成折外预测矩阵供集成层使用，LightGBM 精排层采用用户级时序切分，将每位用户的播放记录按播放时间升序排列后，取各自末尾 10% 的记录作为验证集，其余作为训练集，交互次数少于 5 次的用户不参与验证集构建。集成阶段，精排集成脚本对 DeepFM 与 LightGBM 两路模型统一采用用户级时序切分推断，保证元特征矩阵各行对应同一批验证样本，确保 Meta-LR 训练的 OOF 索引一致。上述时序切分策略模拟了真实推荐场景中用过去行为预估未来偏好的评价语义，使验证集 AUC 能够客观反映模型在实际部署环境下的泛化能力。

## 5.3 MusicMode 推荐算法实现

为实现第四章三路召回、粗排、精排集成与重排的四层管道设计，本节按各层工作分工，逐一完成 ALS 协同过滤召回、FAISS 向量检索召回、热度兜底召回、BST 粗排、LightGBM 与 DeepFM 精排集成以及 MMR 重排的工程代码实现。

### 5.3.1 ALS 协同过滤召回实现

ALS 召回模块依据用户-歌曲历史行为记录构建稀疏交互矩阵，矩阵中仅保留正样本的播放记录，以各用户对同一歌曲的累计播放次数作为非零元值，特征工程阶段生成的负采样记录不参与矩阵构建，避免人工负样本干扰协同信号的提取。训练前以置信度缩放因子  $\alpha=40$  对矩阵进行预乘，以置信度加权机制处理不同频次播放行为的信号强度差异，使高频播放行为在模型学习中占据更高权重，缓解隐式反馈正负样本边界模糊的问题。超参数经 6 组网格搜索实验确定，以 Recall@200 为选优指标，对比了隐向量维度与正则化系数的多组候选后，最终选用 rank=256、reg=0.1、 $\alpha=40$  作为最优配置，并在全量交互矩阵上固定迭代 20 轮完成最终模型训练。推荐生成阶段，系统将每位用户的隐因子向量与全量歌曲隐因子矩阵执行内积计算，按得分降序排列后自动过滤训练集中已出现交互的歌曲，截取前 200 首作为候选集，交由后续粗排层处理。

### 5.3.2 FAISS 向量检索召回实现

FAISS 召回通道利用 DeepFM 训练好的 Embedding 提取歌曲内容语义向量。系统从 DeepFM 模型中抽取歌曲编号、流派、语言、歌手、来源国五类稀疏特征的嵌入，各维度均为 32 维，拼接后得到每首歌曲的 160 维复合向量，同时编码协同过滤信息与内容属性信息。对于训练集中未出现的冷门歌曲，系统将歌曲编号维度置零，仅以内容侧嵌入作为代理向量，使索引覆盖库内全量歌曲而非仅限训练集暖歌曲。对全量歌曲向量执行 L2 归一化后构建内积精确索引（IndexFlatIP），归一化后内积运算等价于余弦相似度，无需额外转换。

构建用户画像向量时，系统按时间分三档对历史播放歌曲的嵌入加权聚合：昨日播放权重 1.0，近 7 日权重 0.6，更早记录权重 0.3。对完播率低于 20%的跳曲施加负权重斥力，将画像主动推离用户排斥的内容方向，高完播率歌曲则追加 1.5 倍权重加成。收藏及歌单中的歌曲亦按对应权重叠加入画像，画像向量归一化后检索前 250 个候选，过滤屏蔽及近期已播歌曲后截取前 200 首，作为候选集交由粗排层处理。

### 5.3.3 热度兜底召回实现

热度兜底召回通道依托 Redis 缓存 30 日歌曲的滚动统计数据构建热度候选池。热度分在对歌曲近 30 日播放量取对数的基础上，叠加热度趋势系数的加权因子，近期播放量快速增长的歌曲可凭趋势系数获得动量加成。系统按热度分降序取前 10000 首构成候选池。召回时，系统依据用户满意度从预设映射表中读取对应的艺术家乘法加成因子，对属于用户偏好艺术家的候选歌曲以热度分与加成因子的乘积重新排序。用户满意度越低时加成因子越大，优先保障偏好艺术家的曝光。满意度越高时因子越小，引导推荐逐步向新艺术家探索。系统同时依满意度动态设定流派过滤范围，不满意时仅在用户偏好度最高的一个流派内筛选候选，满意时放宽至偏好度排名前三的流派范围，非常满意时不作流派限制，在保障偏好与内容多样性之间形成自适应平衡。Redis 读取失败时自动降级至按歌曲流行度字段降序排列的兜底方案，保障通道稳定可用，经排序与过滤后截取前 200 首，作为候选集交由粗排层处理。

### 5.3.4 BST 行为序列 Transformer 粗排实现

BST 粗排层接收三路召回层汇聚的约 600 首候选，通过引入用户历史行为序列建模时序偏好，截取预测概率最高的前 300 首传入精排层。训练阶段，出现次数少于三次的稀疏类别值统一映射为未知标签，防止嵌入层因低频词欠拟合。每条样本由用户最近 50 首历史歌曲编号构成的行为序列与候选歌曲平铺特征共同组成，序列中每首歌曲编号映射为 32 维嵌入并叠加正弦位置编码，目标歌曲嵌入追加至序列末尾后共同送入 3 层 8 头注意力 Transformer 编码器，历史序列编码输出经注意力加权池化聚合为序列表征，与目标嵌入、用户嵌入及稠密特征投影拼接后，经三层全连接网络

逐层附加归一化与 ReLU 激活，由 Sigmoid 输出偏好概率，完整超参数见附表 3 的表 5-7。

数据集采用全局时序切分，末尾 10%划为验证集，前 90%切分 5 折执行折外(OOF)训练，折外预测矩阵与 DeepFM 的 OOF 索引完全对齐，供下游 Stacking 元学习器使用。每折以二分类交叉熵为损失函数，AMP 混合精度训练，CosineAnnealingLR 调度器按余弦曲线周期性衰减学习率，连续 7 轮验证集 AUC 无提升时触发早停，模型序列化至磁盘。推断阶段，系统从用户播放记录中取最近 50 首歌曲编号构成行为序列，不足则前向填充零，与候选歌曲平铺特征拼接后禁用梯度执行前向传播，输出概率降序截取前 300 首进入精排层。行为序列编码、折外训练与粗排推断是本节实现的核心逻辑，关键伪代码见代码 5-1 所示：

代码5-1 BST粗排训练伪代码

```

输入: 平铺特征矩阵 X, 行为序列 seq (B×50), 标签 y, 全局时间戳 ts
输出: bst_model, OOF 预测矩阵 oof_preds

sorted_idx ← argsort(ts) // 全局时序排序

train_idx ← sorted_idx[:0.9N], val_idx ← sorted_idx[0.9N:]

{F1,...,F5} ← split(train_idx, K=5) // 5 折 OOF 划分, 索引与 DeepFM 完全对齐

for k = 1 to 5 do

    fold_model ← BST(seq_len=50, n_layers=3, n_heads=8, d_model=128, dropout=0.45)

    fold_model.fit(X[Ftrain], seq[Ftrain], y, eval=(X[Fk], seq[Fk])) // AMP + CosineAnnealingLR
+ 早停 7 轮

    oof_preds[Fk] ← fold_model.predict(X[Fk], seq[Fk]) // 折外预测存入 OOF 矩阵

end for

Save(oof_preds) // 供 Stacking 元学习器训练使用

bst_model ← BST.fit(X[train_idx], seq[train_idx], y) // 全量训练集训练最终模型

Save(bst_model)

```



```
seq_user ← get_history(user_id)[-50:] // 取最近 50 首，不足则前向补零
scores ← bst_model(X, seq_user) // 禁用梯度，前向传播获取偏好概率
return argsort(scores, descending)[:300] // Top-300 进入精排层
```

### 5.3.5 LightGBM 精排实现

LightGBM 精排层接收 BST 粗排筛选后的 300 首候选，对每首候选歌曲与目标用户构成的样本对打分，与 DeepFM 并行输出偏好概率，由 Meta-LR 元学习器融合后截取前 150 首进入重排层。训练阶段，由用户行为统计、歌曲属性与用户-歌曲交叉匹配等维度特征构成 59 维特征矩阵，其中歌曲重复收听率特征实施 5 折 OOF-TE 折外编码——训练集按全局时间戳划分为 5 折，每折的编码值仅由其余 4 折的正样本比率经贝叶斯平滑计算，以剥离训练标签对该特征的污染，验证集采用全量训练统计值回填。数据集划分采用用户级时序切分，各用户播放记录时序排列后取末尾 10% 为验证集，交互不足 5 次的用户不参与验证集构建。模型以二分类交叉熵为目标函数，连续 800 轮验证集 AUC 无提升时触发早停，记录最优迭代轮次后生成 5 折 OOF 预测矩阵供元学习器训练使用，序列化存储至磁盘，完整超参数配置见附表 3 的表 5-8。推断阶段，推荐服务从特征矩阵中按列索引提取对应的 59 列，调用模型最优迭代轮次推断，输出概率向量供 Meta-LR 集成使用。折外目标编码、早停训练与精排推断是本节实现的核心逻辑，关键伪代码见代码 5-2 所示：

代码5-2 LightGBM精排训练伪代码

```
输入：特征矩阵  $X \in \mathbb{R}^{N \times 59}$ ，标签  $y$ ，训练集索引 train_idx，验证集索引 val_idx
输出：lgbm_model，最优迭代轮次 best_iter

 $\{F_1, \dots, F_5\} \leftarrow \text{split}(\text{train\_idx}, K=5, \text{by}=\text{timestamp})$  // 按时间戳顺序 5 折划分
for k = 1 to 5 do
    song_target_rate[Fk] ←  $(C_{-k}(x) + m \cdot M) / (N_{-k}(x) + m)$ ,  $m=100$  // 折外贝叶斯平滑编码
end for

song_target_rate[val_idx] ←  $(C(x) + m \cdot M) / (N(x) + m)$  // 验证集全量统计回填
```

```
// 二分类交叉熵，AUC 监控早停

lgbm_model ← LightGBM.fit(X_train, y_train, eval=val_set, early_stopping=800)

best_iter ← lgbm_model.best_iteration // 记录最优树数量

Save(lgbm_model, best_iter) // 序列化存储至磁盘

// 推断阶段

X59 ← X66[:, feature_cols] // 对齐训练阶段的 59 维输入列

scores ← lgbm_model.predict(X59, num_iteration=best_iter)

return scores // 概率向量输出至 Meta-LR 元学习器集成
```

### 5.3.6 DeepFM 深度精排实现

DeepFM 精排层接收 BST 粗排筛选后的 300 首候选，对每首候选歌曲与目标用户构成的样本对打分，与 LightGBM 并行输出偏好概率，由 Meta-LR 元学习器融合后截取前 150 首进入重排层。训练阶段，14 个稀疏特征由各独立嵌入表映射为 32 维向量，通过张量拼接融合得到 448 维复合向量，同时送入 FM 层与 DNN 层。FM 层计算稀疏特征间 91 对二阶特征交叉，DNN 层经四层全连接网络逐层提取高阶组合特征，两路输出相加后经 Sigmoid 映射为偏好概率。深度模型输入不含目标编码字段，防止训练标签通过特征路径隐性泄露。

数据集采用全局时序切分，全量样本按全局播放时间戳升序排列，末尾 10% 为验证集，前 90% 切分 5 折执行折外训练，折外预测矩阵供下游 Stacking 元学习器训练使用，划分基准与 BST 完全对齐以保证集成阶段 OOF 索引一致。每折以二分类交叉熵为损失函数，AMP 混合精度训练降低显存峰值，ReduceLROnPlateau 调度器在验证损失停滞时自动衰减学习率，连续 10 轮验证集 AUC 无提升时触发早停，训练完成后模型序列化存储至磁盘，完整超参数配置见附表 3 的表 5-9。推断阶段，特征矩阵转为浮点张量后禁用梯度执行前向传播，输出概率向量供 Meta-LR 集成使用。折外训练、全量建模与精排推断是本节实现的核心逻辑，关键伪代码见代码 5-3 所示：

代码5-3 DeepFM精排训练伪代码

```

输入：特征矩阵 X（14 稀疏+稠密），标签 y，全局时间戳 ts
输出：deepfm_model, OOF 预测矩阵 oof_preds

sorted_idx ← argsort(ts) // 全局时序排序
train_idx ← sorted_idx[:0.9N], val_idx ← sorted_idx[0.9N:]

{F1,...,F5} ← split(train_idx, K=5) // 5 折 OOF 划分，基准与 BST 对齐
for k = 1 to 5 do
    fold_model ← DeepFM(dnn_units=(512,256,128,64), dropout=0.35)
    fold_model.fit(X[F_train], y, eval=(X[Fk], y)) // AMP + ReduceLROnPlateau + 早停 10 轮
    oof_preds[Fk] ← fold_model.predict(X[Fk]) // 折外预测存入 OOF 矩阵
end for

Save(oof_preds) // 供 Stacking 元学习器训练使用

deepfm_model ← DeepFM.fit(X[train_idx], y) // 全量训练集训练最终模型
Save(deepfm_model)

// 推断阶段
X_tensor ← X.float()

scores ← deepfm_model(X_tensor).squeeze() // 禁用梯度，前向传播

return scores // 概率向量输出至 Meta-LR 元学习器集成
    
```

### 5.3.7 Stacking 集成与重排输出的实现

集成层离线阶段，将 DeepFM 与 LightGBM 在 5 折 OOF 训练中各自生成的折外预测概率取索引交集对齐后，按列拼接为二维元特征矩阵，以用户级时序切分的验证集标签训练逻辑回归元学习器，正则化强度 C 设为 1.0，最大迭代 1000 轮，序列化存储至磁盘，元学习器上线后仅执行推断，不产生额外训练开销。推荐生成阶段，精排集成层接收粗排输出的 300 首候选并发送入 DeepFM 与 LightGBM 独立评分，元学习器对两路输出概率构成的元特征矩阵执行预测，取正类概率降序截取前 150 首进入重

排层。重排层依 4.4.3 节的 MMR 策略贪心选曲，对全部候选的集成评分执行 Min-Max 归一化后计算 MMR 值，直至选出 50 首为止。选曲与冷却过滤同步执行，跳过用户近期已播放或显式屏蔽的歌曲，同一艺术家超出上限时追加软惩罚分，上述过滤规则在批处理执行时从数据库实时读取。最终 50 首结果以批量写入方式存入推荐结果表，前端访问时直接从数据库读取渲染，推荐计算与服务层在时序上完全解耦。

Stacking 元学习器训练与 MMR 多样性重排是本节实现的核心逻辑，关键伪代码分别见代码 5-4 与代码 5-5 所示：

代码5-4 Stacking元学习器训练伪代码

```

输入：deepfm_oof (N 维)，lgbm_oof (N 维)，验证集标签 y_train，时间戳 ts
输出：meta_lr，集成权重配置 ensemble_config

// 用户级时序切分，与训练阶段保持一致
train_idx, val_idx ← UserLevelTemporalSplit(ts, val_ratio=0.1, min_interactions=5)

// 取 DeepFM 与 LightGBM 的 OOF 索引交集，确保两路预测严格对齐
common_idx ← intersect(deepfm_oof_idx, lgbm_oof_idx)

deepfm_oof_aligned ← deepfm_oof[position_of(common_idx in deepfm_oof_idx)]
lgbm_oof_aligned ← lgbm_oof[position_of(common_idx in lgbm_oof_idx)]

// 拼接二维元特征矩阵，列顺序：[DeepFM 预测概率, LightGBM 预测概率]
X_meta ← hstack([deepfm_oof_aligned, lgbm_oof_aligned])

y_meta ← y[common_idx]

meta_lr ← LogisticRegression(C=1.0, max_iter=1000)

meta_lr.fit(X_meta, y_meta) // 元学习器训练

w ← meta_lr.coef_ // 提取两路精排权重

Save(meta_lr, w → ensemble_config) // 序列化存储至磁盘

```

代码5-5 MMR多样性重排伪代码

```

输入: ranked (150 首候选及集成评分), song_meta_map, top_n=50

输出: selected (50 首推荐结果)

norm_scores ← MinMaxNorm([s for _, s in ranked]) // 集成评分归一化到[0,1]
remaining ← [sid for sid, _ in ranked] // 待选候选池
selected, sel_vecs, artist_cnt ← [], [], {} // 已选列表、内容向量集、艺术家计数器

while len(selected) < top_n and remaining ≠ ∅ do
    for sid in remaining do
        rel ← norm_scores[sid] // 相关性分
        max_sim ← max cosine_sim(vec[sid], sel_vecs) if sel_vecs else 0 // 多样性惩罚
        // MMR 核心公式:  $\lambda$  权衡相关性与多样性
        mmr_val ← MMR_LAMBDA × rel - (1 - MMR_LAMBDA) × max_sim
        // 艺术家超出上限时施加软惩罚, 避免同艺术家霸榜
        if artist_cnt[artist(sid)] ≥ MAX_PER_ARTIST: mmr_val -= 10.0
        if sid in cooldown_blacklist: skip // 跳过冷却期歌曲
    end for
    best_sid ← argmax(mmr_val) // 贪心选取当轮 MMR 最优曲目
    selected.append(best_sid), remaining.remove(best_sid)
    artist_cnt[artist(best_sid)] += 1 // 更新艺术家计数
    sel_vecs.append(vec[best_sid]) // 将内容向量纳入多样性参照集
end while

return selected

```

## 5.4 MusicWeb 前端实现

本节基于第四章规划的 MVC 五层架构, 完成 MusicWeb 前端平台的架构搭建与各核心功能的工程实现, 使推荐引擎输出的个性化结果能够通过可交互界面完整触达

用户，并通过播放、收藏等行为反馈形成用户画像持续更新的数据闭环。

### 5.4.1 MVC 分层架构搭建

MusicWeb 依 4.1.1 节规划的 MVC 五层架构进行实现，控制层按七个业务域各自部署独立 Servlet 类，解析 HTTP 请求参数后执行登录状态校验并向业务逻辑层转发，不直接操作数据库。对受保护资源访问时，控制层在请求分发前检查 Session 中的登录凭证，未通过者重定向至认证入口，保证接口访问的安全边界。业务逻辑层各 Service 类按业务实体独立拆分，推荐服务组件从推荐结果表读取预计算结果，结合当前收藏状态与近期播放历史对推荐列表执行过滤与补全，向控制层返回可直接渲染的歌曲列表。数据访问层全部 SQL 语句采用参数化查询规避注入风险，RedisUtil 组件在数据库写操作完成后主动使相关缓存键失效，保证读缓存始终反映最新状态。

各业务实体类仅封装字段属性与访问器方法，在控制层、业务逻辑层与数据访问层之间作为统一数据载体流转，避免层间直接依赖松散的键值结构。

### 5.4.2 用户认证与账户管理实现

#### （1）登录实现

登录服务端接收用户名与密码参数，完成非空校验后查询数据库获取账号凭据与状态信息，并对三种状态分别处理：状态正常的账号加载完整用户信息，检查并按需补建默认收藏歌单，依据管理员权限标识跳转至用户主页或管理后台，并将用户对象写入服务端会话；冻结账号重定向至状态说明页并附带冻结原因与截止时间；已注销账号重定向至状态说明页并提示相应信息。用户主动退出时，服务端立即销毁会话并跳转至系统首页。登录页面如图 5-1 所示。



图5-1 系统登录页面

（2）两步式注册实现

注册分为两步引导，如图 5-2 和图 5-3 所示。第一步是服务端对用户名、密码的非空性以及唯一性进行校验，验证通过后将账号基础信息存入用户表中。第二步服务端接收用户勾选过的流派、歌手喜好信息后存入偏好字段，然后开始推荐引擎冷启动过程生成初始推荐候选集，并创建“我喜欢的音乐”默认收藏歌单；以上操作之后立即开始登录工作，用户不需要手动登录就能直接进入主页。



图5-2 用户注册页面（1）

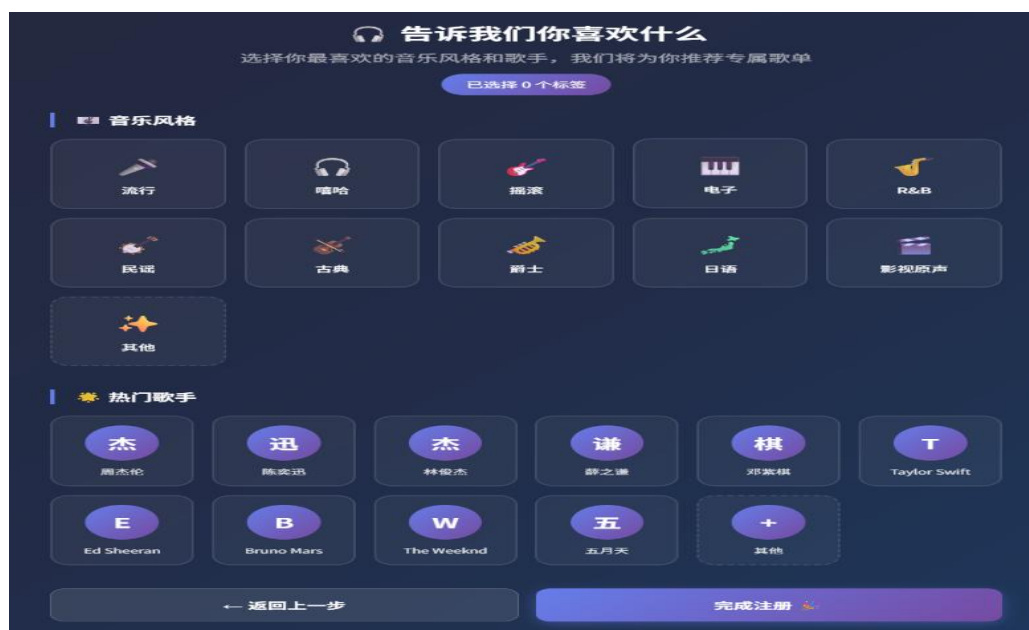


图5-3 用户注册页面（2）

### （3）基本信息与安全设置

账户设置页面包含个人信息、账户安全、账户管理这三个入口，对应基本资料修改、密码变更、账号注销等三项功能，如图 5-4 所示。基本信息修改时，服务端对昵称、邮箱、手机号等字段执行合法性校验，通过后更新数据库相应字段并刷新服务端会话。密码修改时，服务端先检验当前密码是否正确，然后对新密码进行强度校验和唯一性检查，检验结果通过后更新数据库密码字段和重置会话。账号注销需要输入当前密码进行二次确认，确认无误后服务端对账号进行软删除标记，数据库外键约束自动级联删除相关歌单、收藏记录，然后销毁会话并跳转到首页，密码错误或者系统异常的时候保持会话不变并给出错误提示。



图5-4 账户设置页面

### 5.4.3 个性化推荐展示与行为反馈实现

个性化推荐展示界面如图 5-5 所示。推荐数据在 Python 端推荐引擎离线运行之后先存入到数据库推荐结果表中，然后由 Java 端根据需要调用获取并以 JSON 格式发送到前端进行渲染。服务端先检验当前用户的会话是否有效，未登录请求会显示 401 状态码。验证通过后按照综合评分降序读取用户当前推荐列表，每次取 5 首并支持偏移量参数来达到“换一批”的滑动翻页效果，再查询每首歌曲的收藏状态，将歌曲基础信息和收藏状态一起组成 JSON 响应返回给前端。前端推荐卡片展示封面、曲名、演唱者和操作按钮，点击播放后触发完播率隐式采集，而收藏操作则触发评分显式提升，两路反馈数据均写入推荐反馈表，供下次推荐管道调度时消费，构成推荐—行为—更新—再推荐的数据闭环。



图5-5 个性化推荐展示页面

#### 5.4.4 多源音乐检索与在线播放实现

搜索服务端根据关键词和来源参数（网易云/QQ/全部）发送请求给外部接口中间服务，结果如图 5-6 所示。对异质响应结构进行解析，把标题、演唱者、封面及时长这些映射到系统内部的歌曲对象上，选择全部来源时采用交替合并方式达到均衡呈现效果。搜索阶段不会进行数据入库操作，当触发播放或者收藏时，服务端对外部歌曲做两阶段去重处理，去重字段为标题、演唱者和演唱者集合的交集，然后依次调用 P1 到 P5 五个降级元数据补全策略，通过网易云百科、QQ 音乐、Last.fm、MusicBrainz、本地检测的方式保证流派和语种字段完整，为推荐特征提取打下基础。

在线播放使用的是 HTML5 原生能力，可以进行进度控制、模式切换。播放触发后开始前端计时器累计时长。切换歌曲或者关闭页面时，异步请求上报标识和实际时长到服务端，服务端根据这两个信息来计算完播率，并且更新推荐反馈记录。执行收藏操作的时候，外部歌曲经过去重入库以及元数据补齐之后，再由服务端关联到用户默认歌单并清除缓存。被收藏歌曲若是当日推荐的，则将评分提升并置位标记，显式反馈信号随即写入推荐反馈表。



图5-6 音乐搜索结果页面

#### 5.4.5 歌单管理功能实现

歌单服务端用操作类型参数统一处理创建、添加歌曲、更新、删除四种请求，所有操作都先进行当前会话有效性验证，其效果如图 5-7 所示。创建歌单时，服务端使用用户标识以及自定义的名字来创建一个新的歌单记录，将其标记为非默认歌单。当目标歌曲来源于外部平台时，服务端会先进行和 5.5.4 节一样的两阶段去重入库操作，然后检查歌单归属权，符合条件的就将歌曲加入到该歌单中。在修改歌单名称或者描述的时候也要进行归属权的检查，在删除操作时又会拦下对于默认歌单（“我喜欢的音乐”）的删除请求，避免因误删而造成收藏容器的删除，全部写入操作则会用到 JSON 格式返回操作结果。



图5-7 用户歌单管理页面

### 5.4.6 管理员后台功能实现

管理员后台以统一的服务入口接收操作类型参数，将请求路由至系统统计、用户信息管理、申诉管理、歌单管理、歌曲管理五个功能模块，所有请求均先校验会话中的管理员身份标识，非管理员请求直接拦截。

#### （1）系统统计

统计模块聚合数据库中的用户总量、歌曲总量、播放记录数与推荐条目数四项关键指标，展示于数据看板，同时在后台静默执行保护期满 30 天的注销账号物理清理，如图 5-8 所示。

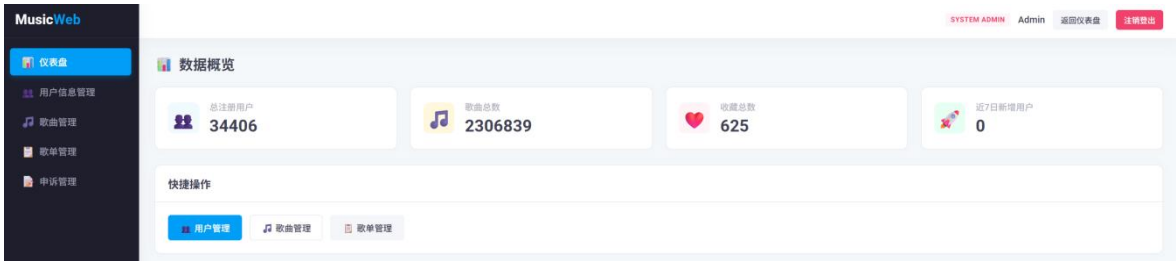


图5-8 管理员后台数据统计看板

#### （2）用户信息管理

用户信息管理模块支持分页查询全部用户列表与单用户详情。账号冻结操作将账

号状态更新为冻结并记录冻结原因与截止日期，而解冻操作则将状态恢复为正常。此外软删除操作将账号标记为已注销并设置 30 天数据保护期，期内数据完整保留。用户申诉审核通过后由申诉模块自动触发解冻或账号恢复，用户信息管理页面如图 5-9 和图 5-10 所示。



图5-9 管理员后台用户信息管理页面



图5-10 管理员后台用户详情页面

### （3）歌曲管理

歌曲管理模块支持关键词分页搜索歌曲列表，可对歌曲执行添加、删除与编辑操作；编辑保存后自动清除服务端歌曲信息缓存，确保前端数据一致性。管理员还可在列表中直接预览播放歌曲，验证音频文件可用性，如图 5-11 所示。



图5-11 管理员后台歌曲管理页面

(4) 歌单管理

歌单管理模块以全局视角展示系统内所有用户的歌单列表，每条记录附带创建者信息与歌曲数量。管理员可查看任意歌单的歌曲明细，对违规歌单执行删除操作或从歌单中移除特定歌曲。系统对默认歌单（"我喜欢的音乐"）设有保护，禁止管理端强制删除用户的默认歌单，如图 5-12 所示。

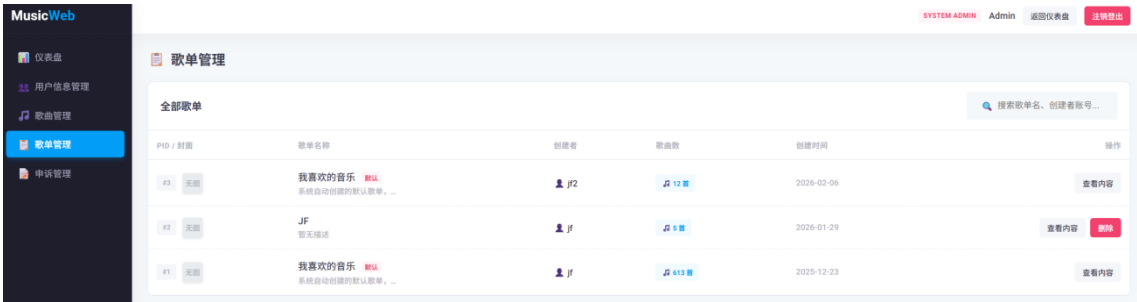


图5-12 管理员后台歌单管理页面

(5) 申诉管理

申诉管理模块展示全部用户提交的申诉列表及详情，含申诉类型、原因与联系邮箱。批准申诉时，服务端修改申诉状态、解除账号冻结或注销标记，并通过异步线程向用户联系邮箱发送审核通过通知。而拒绝申诉时则同步更新状态并发送拒绝通知邮件，如图 5-13 所示。

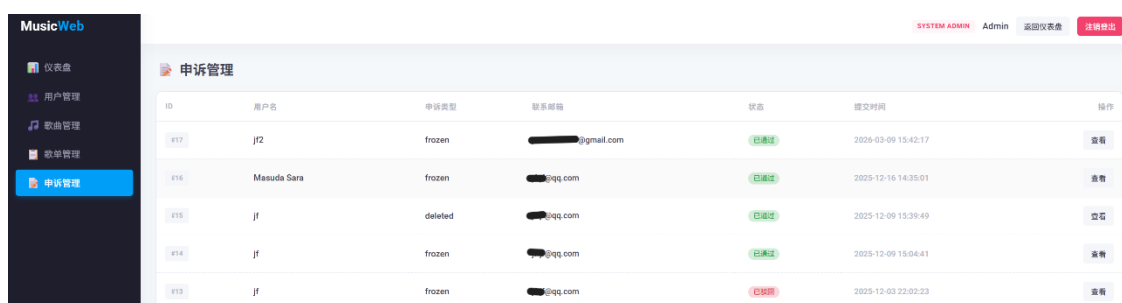


图5-13 管理员后台申诉管理页面

## 5.5 本章小结

本章沿第四章设计方案完成从原始数据到线上推荐服务的全链路工程落地。数据层以 PySpark 对 229 万首歌曲与 737 万条行为交互记录执行分布式 ETL，并结合列裁剪与谓词下推技术，将磁盘 I/O 开销进行大幅削减。经特征工程，系统为 BST 粗排、LightGBM 精排与 DeepFM 精排三个模型分别构建了 66 维、59 维及 66 维的独立特征子集，覆盖用户画像、歌曲属性、行为统计、交叉匹配、时序动态与 SVD 隐向量嵌入六个维度。算法管道按四层架构顺序落地，三路召回汇聚 600 余首候选，BST 以历史行为序列建模时序偏好粗排至 300 首，DeepFM 与 LightGBM 双模型并行精排，将折外 OOF 矩阵送入逻辑回归元学习器融合至 150 首，最终由 MMR 重排结合冷却过滤输出 50 首，批量写入推荐结果表。前端 MusicWeb 基于 MVC 五层架构落地用户认证、推荐展示、多源检索与管理后台，完播率隐式采集与收藏显式反馈双路信号持续回写推荐反馈表，驱动下一轮管道调度。推荐计算与前端服务时序解耦，页面请求直读数据库渲染，响应延迟不受离线计算影响。至此，本系统的主体工程已全线贯通，后续第六章将结合离线量化指标，对推荐体系的实际应用效果进行验证和分析。

## 6 系统测试与评估

第五章完成了四层推荐管道与 MusicWeb 前端平台的全量工程实现，系统已具备向真实用户提供个性化推荐服务的能力。但工程实现是否真正有效，需要以量化指标加以验证。为此，本章以 KKBOX 数据集为基准，从纵横两个方向展开离线评估：纵向通过五阶段递进消融实验，以随机基准为起点，逐步叠加热度召回、BST 单模型排序、精排集成与 MMR 重排，量化各模块对排序质量的实际贡献。横向则在同一验证集上对 BST 粗排层、DeepFM 精排层、LightGBM 精排层与 Meta-LR 集成模型以 AUC 为统一基准进行横向比较，考察 Stacking 集成架构相对于各单体模型的判别能力优势。

### 6.1 评估指标与实验设置

#### 6.1.1 评估指标说明

离线评估选取命中率（HR@K）、精确率（Precision@K）、归一化折损累积增益（NDCG@K）、平均倒数排名（MRR）列表内多样性（Shannon 熵）与曲下面积（AUC）六项指标，从不同角度量化推荐结果的排序质量与覆盖多样性，前四项均在 Top-5 推荐场景下计算。

##### (1) 命中率

命中率衡量系统能否将用户真正感兴趣的内容纳入推荐列表前 K 位，即验证集中至少命中一个正样本的用户占比，命中率越高说明系统对绝大多数用户均能在有限推荐位中呈现其偏好内容，公式见式（6-1）所示。

$$HR@K = \frac{1}{|U|} \sum_{u \in U} 1[\text{rank}_u \leq K] \quad (6-1)$$

式中 U——为验证集用户集合；

$|U|$ ——为用户总数；

$\text{rank}_u$ ——为用户 u 的真实正样本在推荐列表中的排名位置；

$1[\cdot]$ 为示性函数，当括号内条件成立时取值为 1，否则为 0。



## (2)精确率

精确率衡量推荐列表中相关内容所占的比例，精确率越高说明系统推荐的内容中  
有用信息密度越大，用户无需在大量无关推荐中筛选目标，在音乐推荐场景下直接关  
联用户对推荐列表的整体满意度，公式见式（6-2）所示。

$$\text{Precision@K} = \frac{1}{|U|} \sum_{u \in U} \frac{R_u \cap T_u}{K} \quad (6-2)$$

式中  $R_u$ ——为系统为用户  $u$  生成的 Top-K 推荐列表；

$T_u$ ——为用户  $u$  的真实正样本集合；

$K$ ——推荐列表长度，本系统取  $K=5$ 。

## (3)归一化折损累积增益

归一化折损累积增益在命中基础上引入排名位置的对数折损权重，对排名靠前的  
命中给予更高奖励，能够区分"排在第 1 位命中"与"排在第 10 位命中"的质量差异，  
更精细地反映系统将偏好内容优先排列于列表前端的能力，计算公式见式（6-3）和  
（6-4）所示。

$$\text{NDCG@K} = \frac{1}{|U|} \sum_{u \in U} \frac{DCG_u}{IDCG_u} \quad (6-3)$$

$$DCG_u = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)} \quad (6-4)$$

式中  $IDCG_u$ ——为用户  $u$  在理想排序下的最大 DCG 值，即将该用户所有正样本置  
于列表最前端时所能达到的 DCG 上界；

$DCG_u$ ——用户  $u$  的折损累积增益；

$rel_i$ ——第  $i$  个推荐位置的相关性标签，命中正样本时取值为 1，否则为 0；

$\log_2(i+1)$ ——位置折损因子，使排名越靠后的命中贡献以对数速率递减；

$K$ ——推荐列表长度，本系统取  $K=5$ 。

## (4) 平均倒数排名

平均倒数排名以每位用户正样本首次出现位置的倒数取均值，对列表头部极为敏  
感，正样本出现在第 1 位贡献为 1.0，出现在第 5 位仅为 0.2，MRR 高的系统不仅能

命中用户兴趣，还能将最相关内容优先呈现，公式见式（6-5）所示。

$$MRR = \frac{1}{|U|} \sum_{u \in U} \frac{1}{rank_u} \quad (6-5)$$

式中  $rank_u$ ——为用户  $u$  的正样本在推荐列表中首次出现的排名位置；

$|U|$ ——为验证集用户总数。

#### （5）列表内多样性

列表内多样性以推荐列表中流派分布的 Shannon 熵为度量，熵值越高说明推荐结果覆盖的音乐风格越丰富，用户不会陷入单一流派的信息茧房。对每位用户的 Top-K 推荐列表统计各流派占比后计算熵值，再对所有用户取均值，公式见式（6-6）所示。

$$H = - \sum_{c \in C} p_c \log_2 p_c \quad (6-6)$$

式中  $H$ ——单用户推荐列表的流派 Shannon 熵；

$C$ ——推荐列表中出现的流派集合；

$p_c$ ——流派  $c$  的歌曲数占推荐列表总长度的比例；

$\log_2$ ——以 2 为底的对数，使熵值单位为比特。

#### （6）曲线下面积

AUC（Area Under the ROC Curve）为 ROC 曲线下面积，衡量模型区分正负样本的全局排序能力。其含义为：从正样本集与负样本集中各随机取一个样本，模型将正样本得分排于负样本之上的概率，AUC 越高说明模型的二分类判别能力越强，计算公式见式（6-7）所示。

$$AUC = \frac{1}{|P| \cdot |N|} \sum_{p \in P} \sum_{n \in N} 1[f(p) > f(n)] \quad (6-7)$$

式中  $P$ ——正样本集合；

$N$ ——负样本集合， $|P|$ 与 $|N|$ 分别为对应集合的样本数量；

$f(\cdot)$ ——模型对样本的预测得分函数；

$1[\cdot]$ ——示性函数，当括号内条件成立时取值为 1，否则为 0。

### 6.1.2 实验设置

本文设置两组离线实验，使用 KKBOX 数据集，涵盖 28172 名用户约 73 万条交互记录，按用户级时序切分策略划分验证集——各用户播放记录升序排列后取末尾 10% 构成验证正样本集，其余用于训练，使验证语义与真实推荐场景中“以历史预测未来”的评价逻辑保持一致。

消融实验采用五阶段递进设计，以量化拆解管道内各层的实际增益。第一阶段以均匀随机评分替代一切有效信号，作为无信息下界，排除任何排序先验。第二阶段引入热度通道召回，以歌曲全局累计播放量为评分依据，代表仅凭热度先验的候选推荐能力。第三阶段在热度召回产出的候选集上叠加 BST 单模型排序，以行为序列 Transformer 的预测概率对候选重新排列，评估序列建模在热度召回基础上的单独增益。第四阶段引入完整精排集成，Meta-LR 元学习器融合 DeepFM 与 LightGBM 两路折外预测概率，对候选集输出最终集成评分。第五阶段在精排集成结果上施加 MMR 多样性重排，构成四层管道的完整运行配置，且额外引入推荐列表流派 Shannon 熵量化多样性，并进一步考察 MMR 权重系数  $\lambda$  在 0.4 至 1.0 范围内变化时的精度与多样性权衡关系，为系统  $\lambda$  设计点的选取提供定量依据。各阶段以 HR@5、Prec@5、NDCG@5 与 MRR 四项指标度量作为评判标准。

横向对比实验聚焦于各模型在验证集上的正负样本二分类判别能力，参评的四个模型按管道层级排列：BST 粗排层、单模型 DeepFM 精排层、LightGBM 精排层与 Meta-LR 集成模型，统一以 AUC 为量化基准。三路召回通道输出的均是非概率性质的相对评分，无法直接绘制 ROC 曲线，仅在消融阶段参与排序指标评估，不纳入 AUC 横向对比。Meta-LR 的预测概率严格取自 5 折时序交叉验证的折外预测(OOF)得分，以规避集成训练中的标签泄漏，使 AUC 结果客观反映模型对未知样本的泛化水平。

## 6.2 消融实验结果分析

本实验的研究目标是量化四层推荐管道中各模块对排序质量的实际贡献，验证分层叠加设计的有效性。各层逐步引入后，排序指标应呈现持续正向增益。实验采用五阶段递进叠加配置，以随机基准为起点，依次叠加热度召回、BST 粗排、精排集成与

MMR 重排，通过逐层消融对比量化每一环节的贡献，结果如图 6-1 和图 6-2 所示。

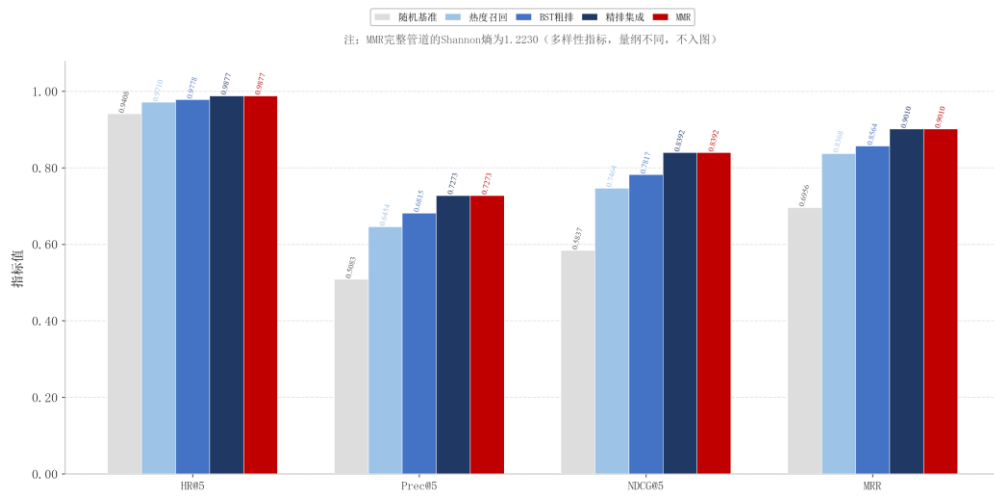


图6-1 四层管道指标得分柱状图

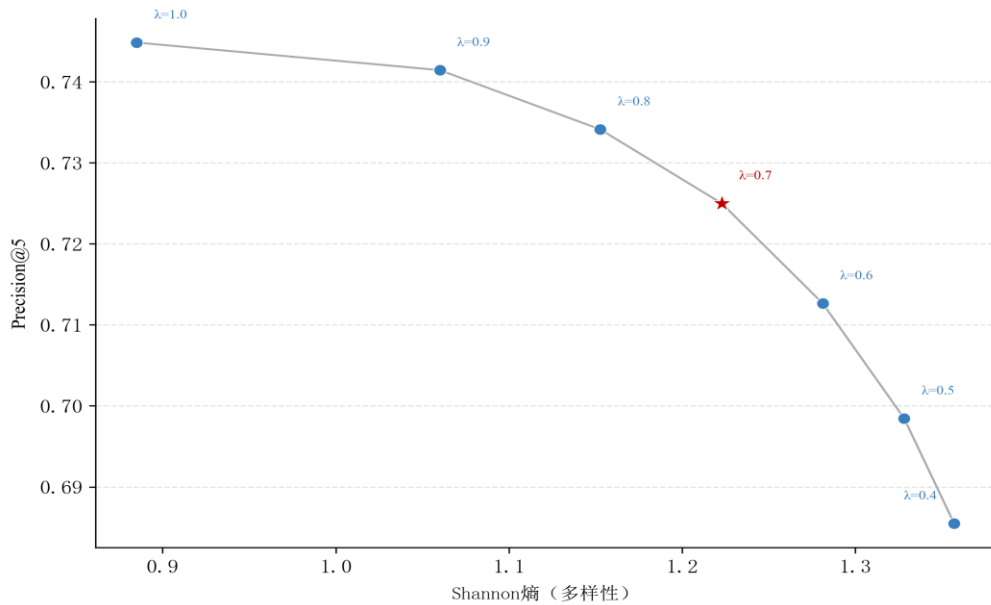


图6-2 MMR帕累托曲线

从图 6-1 可以看出，五阶段配置的差异主要集中在  $\text{Prec}@5$ 、 $\text{NDCG}@5$  和  $\text{MRR}$  三项排序指标上。随机基准下  $\text{HR}@5$  已达 0.9408，说明随机召回对绝大多数用户均能覆盖至少一首正样本，但  $\text{Prec}@5$ 、 $\text{NDCG}@5$  与  $\text{MRR}$  仅分别为 0.5083、0.5837 和 0.6956，精准排序能力几近于零，构成各配置的绝对下界。引入热度召回后，上述三项指标出现五阶段内大幅度的单步跃升， $\text{Prec}@5$  达 0.6454， $\text{NDCG}@5$  达 0.7464，

MRR 达 0.8368，热度先验在无序列建模介入的条件下已能为候选集注入显著的相关性导向。叠加 BST 单模型排序后各项指标持续正向，Prec@5、NDCG@5 与 MRR 分别升至 0.6815、0.7817 和 0.8564，增量方向稳定但幅度较热度阶段有所收敛。精排集成阶段的单步增益最为突出，Prec@5、NDCG@5 与 MRR 分别提升至 0.7273、0.8392 和 0.9010，HR@5 升至 0.9877，Meta-LR 将 DeepFM 与 LightGBM 两路互补评分融合后，正样本向列表头部集中的程度明显高于单一模型阶段。MMR 重排后四项排序指标与精排集成保持一致，多样性贡献体现于推荐列表流派 Shannon 熵，均值达 1.2230 比特。图 6-2 的帕累托曲线呈现了  $\lambda$  从 1.0 降至 0.4 时精度与多样性的权衡关系， $\lambda=1.0$  时 Shannon 熵仅 0.88，随  $\lambda$  降低多样性持续上升，曲线在  $\lambda=0.7$  附近出现明显拐点，此后多样性提升趋于平缓而精度损失加快， $\lambda=0.7$  因此被选为系统设计点。

综合各阶段来看，热度召回奠定了排序质量的基础，精排集成贡献了最大的头部命中提升，BST 粗排提供了稳定增量，MMR 重排在不损失精度的条件下补充了流派多样性，四层管道各环节均有实质性贡献，分层设计的有效性得到验证。

### 6.3 各模型 AUC 对比实验

消融实验已从排序质量层面验证了四层管道的整体有效性，本实验进一步聚焦精排层，验证基于 OOF 折外框架的集成策略能否将 DeepFM 与 LightGBM 的异质建模能力转化为二分类判别性能上的互补增益。横向对比 BST 粗排层、单模型 DeepFM 精排层、LightGBM 精排层与 Meta-LR 精排集成在验证集上的 AUC，结果如图 6-3 所示。

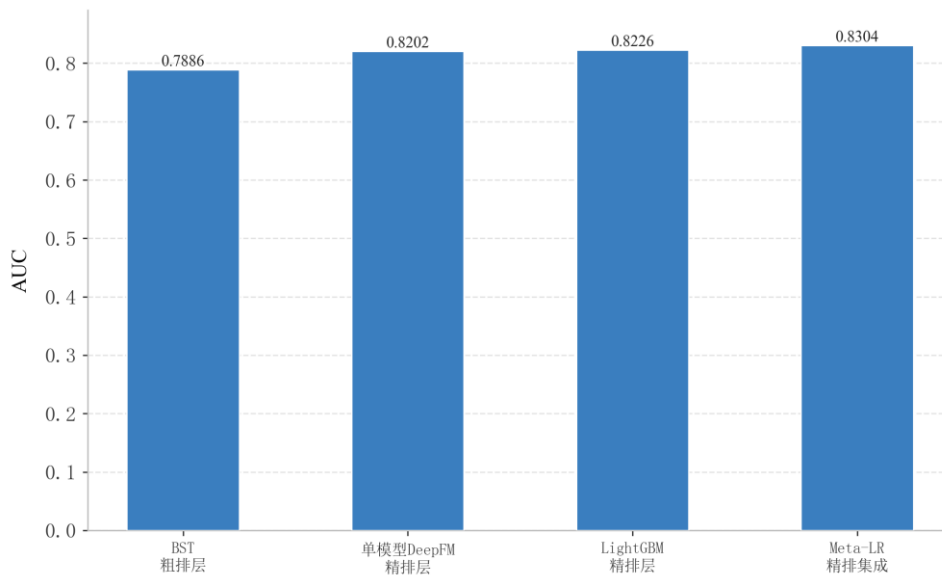


图6-3 各模型AUC得分

从图中可以看出，各模型 AUC 与其建模方式和管道层级之间有显著对应关系。BST 粗排层 AUC 为 0.7886，在四个参评模型中最低，Transformer 的自注意力机制须依赖密集的历史行为序列才能有效感知偏好规律，然而 KKBOX 数据集中大量用户交互次数有限，序列有效长度受压，其注意力机制优势因此无从发挥，二分类判别能力受到制约。单模型 DeepFM 精排层 AUC 为 0.8202，其因子分解机组件与深度网络联合训练，在不依赖手工特征组合的前提下同时捕获低阶与高阶特征交叉，对推荐场景中大量稀疏类别特征具有较好的泛化能力，在 KKBOX 的特征结构下得到了较充分的体现。LightGBM 精排层 AUC 为 0.8226，与 DeepFM 仅相差 0.0024，二者判别能力实质上处于同一水平，但前者的微幅领先与其 59 维手工构建特征密切相关，用户行为统计、热度衰减与流派偏好等聚合信号为梯度提升树提供了扎实的判别基础，对离散统计特征的显式利用弥补了 DeepFM 在这一维度上的不足，两种精排模型从不同建模视角形成互补。Meta-LR 精排集成以 0.8304 居首，集成增益主要来自 DeepFM 与 LightGBM 在错误样本上的互补性，逻辑回归元学习器通过 OOF 折外框架学习两者的互补误差分布，使整体判别能力优于任一单独模型。

综上所述，BST 在粗排阶段的初步过滤上表现符合预期，DeepFM 在处理稀疏特征高阶静态交叉时展现出较强的泛化能力，LightGBM 则凭借其构建的聚合特征在判

别能力上与前者旗鼓相当，两者在不同建模维度上各有侧重。这也是本研究在精排层引入集成设计的初衷，其核心考量并非单纯追求全量样本下 AUC 绝对值的攀升，而是将 DeepFM 对稀疏特征的隐式交叉建模能力与 LightGBM 对聚合统计特征的显式利用进行融合互补。从 6.3 节消融实验各项排序质量指标的实际涨幅来看，这种多视角特征融合策略切实优化了列表头部的推荐精度，从数据层面支撑了本系统多层级推荐管道架构的有效性。

## 6.4 本章小结

本章围绕四层推荐管道实施了两组离线验证，以六项指标构建统一量化基准。消融实验通过五个递进配置逐层验证了各模块贡献，其中热度召回对排序质量的奠基贡献最为显著，BST 单模型排序提供了稳定的增量，精排集成为管道内单步提升幅度最大的环节，MMR 重排在排序精度不变的前提下有效拓宽了推荐列表的流派覆盖范围，帕累托曲线确认  $\lambda=0.7$  为精度与多样性之间的合理设计点，四层管道各环节均产生了可量化的正向贡献。

AUC 对比实验中，Meta-LR 精排集成的二分类判别能力居四个参评模型之首，DeepFM 与 LightGBM 在精排层旗鼓相当并从稀疏特征交叉与聚合统计特征建模两个维度形成互补，BST 受制于 KKBOX 数据集稀疏性，序列建模优势未能充分释放。Meta-LR 通过 OOF 折外框架融合两路互补误差分布，整体判别能力优于任一单独模型，与消融实验的排序质量增益相互印证，共同验证了本系统多层级推荐管道设计的合理性。

## 结 论

本文根据 KKBOX 公开数据集，设计并实现了一套覆盖四层推荐管道的个性化音乐推荐系统原型，采用三路召回、粗排、精排集成、重排四个阶段，Meta-LR 精排集成在验证集上的 AUC 达到 0.8304，达到预期工程目标。

### （1）主要工作总结

对于如何实现千人千面个性化推荐的目标，本文从数据、算法与交互三个维度展开论述。数据层面，以 KKBOX 数据集为基础构建 ETL 管道，结合 OOF 目标编码策略消除高基数类别特征的标签泄漏风险，为每位用户提炼出涵盖行为统计、流派偏好与时序趋势的个人画像特征，使模型能够捕捉用户偏好的细粒度差异。算法层面，四层管道针对不同用户状态提供差异化的推荐路径，三通道并联召回保障活跃用户与冷启动用户均能获得有效候选，BST 在粗排阶段完成候选初筛，DeepFM 与 LightGBM 分别从静态特征深度交叉和梯度提升工程特征两个互补视角精排，经 Meta-LR 融合后再由 MMR 重排主动调节流派多样性，避免推荐结果趋于同质。交互层面，MusicWeb 前端通过事件驱动的异步上报机制采集用户播放完成率与评分行为，持续回写推荐系统，构建“推荐—反馈—优化”的闭环，使用户画像随使用深度不断丰富。

### （2）创新点

本文在个性化音乐推荐系统的设计与实现过程中，分别从方法策略、应用实践以及系统工程三个维度展开了创新性工作。在方法策略层面，提出基于离线 Stacking 框架的异构集成推荐策略，采用 5 折 OOF 编码消除特征泄漏隐患，结合 BST 粗排初筛，并在精排层融合 DeepFM 的深度交叉与 LightGBM 的工程建模能力，有效提升了正负样本的判别精度与系统泛化性。在应用实践层面，针对音乐推荐中普遍存在的“信息茧房”与列表同质化痛点，创造性地在四层推荐管道末端部署了最大边际相关（MMR）算法，通过对流派与艺术家分布施加多样性软约束，在保障个性化匹配质量的前提下显著提升了推荐结果的丰富度，为解决长尾曲目分发与内容多样性失衡提供了有效方案。在系统工程层面，本文设计并实现了一套双模块解耦的完整系统原型，



由基于 Java Web 的交互模块 MusicWeb 与基于 Python 的算法引擎 MusicMode 协同构成，依托异步批处理机制与完播率反馈闭环，实现了从分布式 ETL 治理到前端交互反馈的全链路工程落地，为中小型音乐平台的个性化服务构建提供了可参考的闭环架构方案。

### （3）不足与展望

本研究存在以下几点不足：系统部署于单机环境，缺乏生产级别的高可用保障；BST 的序列建模能力受制于 KKBOX 数据集的高稀疏性，Transformer 的长程依赖优势未能充分发挥；冷启动用户的推荐仍依赖注册偏好与热度兜底，画像精细度与活跃用户相比尚有差距；离线指标与真实用户体验之间的映射关系缺乏大规模在线实验的支撑。

后续可从以下方向推进：引入图神经网络对用户-歌曲交互关系建模，或以对比学习方法缓解数据稀疏问题；将系统迁移至微服务与容器化架构，结合实时特征流实现推荐的动态更新；建立在线 A/B 测试框架，以点击率、留存率等真实业务指标对离线评估体系加以验证与补充。

## 参考文献

- [1]赵吉, 胡海然, 张婷. 基于 Spark 的个性化音乐推荐系统设计与实现[J]. 电子制作, 2024(18): 14-15.
- [2]杨率. 基于深度神经网络的个性化实时音乐推荐系统研究与实现[D]. 武汉: 华中科技大学, 2023: 3-69
- [3]李津. 基于知识图谱的个性化音乐推荐系统设计与实现[J]. 科学技术创新, 2024(02): 127-130.
- [4]冯亚寒. 基于 WSM-CNN 及其改进的音乐多情感分类方法研究与应用[D]. 北京: 北京建筑大学, 2025: 3-49
- [5]吴亚迪, 陈平华. 基于用户长短期偏好和音乐情感注意力的音乐推荐模型[J]. 广东工业大学学报, 2023, 40(04): 37-44.
- [6]王慧心, 童向荣. 融合知识图谱的推荐系统研究进展[J]. 浙江大学学报(工学版), 2023, 57(08): 1527-1540.
- [7]梁锋, 羊恩跃, 潘微科等. 基于联邦学习的推荐系统综述[J]. 中国科学: 信息科, 2022, 52(05): 713-741.
- [8]郑楠, 章颂, 刘玉桥等. 基于深度学习的群组推荐方法研究综述[J]. 自动化学报, 2024, 50(12): 2301-2324.
- [9]闫猛猛, 汪海涛, 贺建峰等. 基于自监督学习的序列推荐算法[J]. 重庆邮电大学学报(自然科学版), 2023, 35(04): 722-731.
- [10]龚雪鸾, 陈艳姣, 王帅. 在线广告点击率预测方法的研究综述[J]. 中文信息学报, 2023, 37(04): 1-17.
- [11]Liu Z, Xu W, Zhang W, et al. An emotion-based personalized music recommendation framework for emotion improvement [J]. Information Processing & Management, 2023, 60(3): 103256.
- [12]WANG D, ZHANG X, YIN Y, et al. Multi-view enhanced graph attention network for session-based music recommendation[J]. ACM Transactions on Information Systems, 2023, 42(1): 16.

- [13]Li L, Tao D, Zheng C, et al. Attentive autoencoder for content-aware music recommendation[J]. CCF Transactions on Pervasive Computing and Interaction, 2022, 4(1): 76-87.
- [14]Mao Y, Zhong G, Wang H, et al. Music-CRN: An efficient content-based music classification and recommendation network[J]. Cognitive Computation, 2022, 14(6): 2306-2316.
- [15]Paul S, Singh S, Rajbhoj S. Personalized music recommendation system [EB/OL]. (2021-01) [2026-03-18]. <https://ssrn.com/abstract=3772631>.
- [16]Niu X, Wang J, Chen X. Collaborative filtering-based music recommendation in Spark [J]. Mathematical Problems in Engineering, 2022: 1-12.
- [17]Johnson J, Douze M, Jegou H. Billion-scale similarity search with GPUs [J]. IEEE Transactions on Big Data, 2021, 7(3): 535-547.
- [18]Wang X, Li Q, Yu D, et al. Neural causal graph collaborative filtering [J]. Information Sciences, 2024, 680: 120872.
- [19]TRAN V, SALHA-GALVAN G, HENNEQUIN R, et al. Transformers meet ACT-R: repeat-aware and sequential listening session recommendation[C]// Proceedings of the 18th ACM Conference on Recommender Systems. New York: ACM, 2024: 486-496.
- [20]CHEN Q, ZHAO H, LI W, et al. Behavior sequence transformer for e-commerce recommendation in Alibaba[C]// Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data. New York: ACM, 2019: 1-4.

## 致 谢

总觉来日方长，毕业无期，行文至此，也百感交集。

人生中有许多节点，有些走得轻盈，有些却走得深重。这两年，无疑是我求学轨迹里权重最大的一段。从 2025 年冬日敲下第一行字，到如今数月间的字斟句酌、反复推敲，其间的甘苦，犹如一次漫长且充满未知的模型训练。二十四岁，不算年轻，却也不算老，刚好是一个人应当学着对自身命运按下“回车键”的年纪。这篇论文，便是我在这个路口，交出的一份最为诚恳的答卷。

最深的谢意，首先要献给我的指导老师王波老师。在这场漫长的学术马拉松里，无论是线上的探讨还是线下的指点，王老师从未缺席。从最初选题时帮我锚定方向，到系统框架构建中的反复斟酌，再到初稿里那些我自以为严密、实则逻辑断层的段落，王老师的每一次批注都能直击要害，绝不以空洞的套话敷衍。学问之道，贵在真诚与严谨。正是王老师那如同底层代码般扎实而耐心的引导，才让这篇论文得以拨开冗杂的噪点，一步步走向清晰。师恩如海，我将永远铭记。

感谢广州华立学院计算机工程学院的每一位任课老师。两年的光阴里，算法、数据库、分布式计算、机器学习……这些名词在黑板上出现时，我未必真切地感受到它们的重量。然而，当我真正坐在屏幕前，开始搭建这套个性化音乐推荐系统时，我才陡然发觉，那些曾经以为枯燥的知识，早已悄无声息地沉入地基，支撑起了整个系统的每一处细节。知识的给予往往是无声的，却总能在你最需要的时候，化作最坚实的算力。

最深沉的感激，要留给我的家人。他们或许永远弄不懂什么是协同过滤，什么是推荐算法，也看不明白终端里跳动的代码，但这从未妨碍他们给予我百分之百的爱。十七年的求学路，每一次跌倒与迷茫，都有他们在身后稳稳托底。他们不苛求我取得多耀眼的成绩，只求我平安顺遂。这份爱，不张扬、不喧哗，却是我人生长河里最不可或缺的前行动力，也是我越过所有难关时最踏实的依靠。

行文至此，这段学生时代的故事即将画上句点。但这并非终局，只不过是跳出了当前的循环，准备去往一个更广阔的天地重新运行。二十四岁，往后的路还长，愿我能走得踏实，走得辽阔，带着这十七年积攒的所有勇气，去迎接真实世界里更为庞杂的数据洪流。

欲买桂花同载酒，终不似，少年游。谨以此篇，献给所有陪我走过这段时光的人。

## 附 录

附表 1:

表 4-1 用户表结构

| 字段名               | 数据类型         | 约束                | 说明                    |
|-------------------|--------------|-------------------|-----------------------|
| id                | INT          | PK,AUTO_INCREMENT | 用户主键                  |
| username          | VARCHAR(50)  | UNIQUE NOT NULL   | 用户名                   |
| password          | VARCHAR(255) | NOT NULL          | 密码                    |
| email             | VARCHAR(100) | —                 | 邮箱                    |
| nickname          | VARCHAR(50)  | —                 | 昵称                    |
| phone             | VARCHAR(20)  | —                 | 手机号                   |
| status            | ENUM         | NOT NULL          | active/frozen/deleted |
| frozen_until      | TIMESTAMP    | —                 | 冻结截止时间                |
| frozen_reason     | VARCHAR(255) | —                 | 冻结原因                  |
| deleted_at        | TIMESTAMP    | —                 | 逻辑删除时间                |
| preferred_genres  | VARCHAR(255) | —                 | 偏好流派                  |
| preferred_artists | VARCHAR(255) | —                 | 偏好艺术家                 |
| create_time       | TIMESTAMP    | DEFAULT NOW()     | 注册时间                  |

表 4-2 歌曲表结构

| 字段名       | 数据类型         | 约束                | 说明                  |
|-----------|--------------|-------------------|---------------------|
| id        | INT          | PK,AUTO_INCREMENT | 歌曲主键                |
| title     | VARCHAR(255) | NOT NULL          | 歌名                  |
| artist    | VARCHAR(255) | —                 | 歌手                  |
| album     | VARCHAR(255) | —                 | 专辑                  |
| duration  | INT          | —                 | 时长（秒）               |
| genre     | VARCHAR(100) | —                 | 流派                  |
| genre_ids | VARCHAR(255) | —                 | KKBOX 原始流派<br>ID 列表 |

表 4-2（续表） 歌曲表结构

| 字段名            | 数据类型         | 约束                | 说明          |
|----------------|--------------|-------------------|-------------|
| id             | INT          | PK,AUTO_INCREMENT | 歌曲主键        |
| language       | VARCHAR(10)  | —                 | 语言代码        |
| release_year   | INT          | —                 | 发行年份        |
| kkbox_id       | VARCHAR(100) | —                 | KKBOX 原曲 ID |
| popularity     | INT          | DEFAULT 0         | 歌曲热度        |
| origin_country | VARCHAR(50)  | —                 | 原产地         |
| cover_image    | VARCHAR(500) | —                 | 封面图片路径      |

表 4-3 播放历史表结构

| 字段名          | 数据类型         | 约束                | 说明          |
|--------------|--------------|-------------------|-------------|
| id           | INT          | PK,AUTO_INCREMENT | 歌曲主键        |
| language     | VARCHAR(10)  | —                 | 语言代码        |
| release_year | INT          | —                 | 发行年份        |
| kkbox_id     | VARCHAR(100) | —                 | KKBOX 原曲 ID |

表 4-4 用户歌单表结构

| 字段名         | 数据类型         | 约束                   | 说明       |
|-------------|--------------|----------------------|----------|
| id          | INT          | PK,AUTO_INCREMENT    | 主键       |
| user_id     | INT          | FK→users.id NOT NULL | 创建者 ID   |
| name        | VARCHAR(100) | NOT NULL             | 歌单名称     |
| description | TEXT         | —                    | 歌单描述     |
| cover_image | VARCHAR(500) | —                    | 歌单封面     |
| is_default  | TINYINT(1)   | DEFAULT 0            | 是否为默认收藏夹 |
| create_time | TIMESTAMP    | DEFAULT NOW()        | 创建时间     |
| update_time | TIMESTAMP    | DEFAULT NOW()        | 最后更新时间   |

表 4-5 歌单歌曲关联表结构

| 字段名         | 数据类型      | 约束                               | 说明    |
|-------------|-----------|----------------------------------|-------|
| id          | INT       | PK,AUTO_INCREMENT                | 主键    |
| playlist_id | INT       | FK→user_playlists.id<br>NOT NULL | 歌单 ID |
| song_id     | INT       | FK→songs.id NOT NULL             | 歌曲 ID |
| add_time    | TIMESTAMP | DEFAULT NOW()                    | 添加时间  |

表 4-6 推荐结果表结构

| 字段名         | 数据类型        | 约束                   | 说明      |
|-------------|-------------|----------------------|---------|
| id          | INT         | PK, AUTO_INCREMENT   | 主键      |
| user_id     | INT         | FK→users.id NOT NULL | 目标用户 ID |
| song_id     | INT         | FK→songs.id NOT NULL | 推荐歌曲 ID |
| score       | DOUBLE      | —                    | 集成推荐得分  |
| source_type | VARCHAR(20) | DEFAULT 'deepfm'     | 推荐来源标记  |
| create_time | DATETIME    | DEFAULT NOW()        | 推荐生成时间  |

表 4-7 推荐反馈表结构

| 字段名            | 数据类型       | 约束                      | 说明    |
|----------------|------------|-------------------------|-------|
| id             | INT        | PK,AUTO_INCREMENT       | 主键    |
| user_id        | INT        | FK→users.id NOT NULL    | 用户 ID |
| song_id        | INT        | FK→songs.id NOT<br>NULL | 歌曲 ID |
| recommend_date | DATE       | NOT NULL                | 推荐日期  |
| was_played     | TINYINT(1) | DEFAULT 0               | 是否已播  |



表 4-7(续表) 推荐反馈表结构

| 字段名                     | 数据类型       | 约束            | 说明             |
|-------------------------|------------|---------------|----------------|
| play_completion         | FLOAT      | —             | 播放完成度<br>(0~1) |
| was_favorited           | TINYINT(1) | DEFAULT 0     | 是否已收藏<br>(0/1) |
| consecutive_ignore_days | INT        | DEFAULT 0     | 连续忽略天<br>数     |
| feedback_score          | FLOAT      | DEFAULT 0     | 综合反馈得<br>分     |
| cooldown_until          | DATE       | —             | 推荐冷却期<br>截止日   |
| created_at              | TIMESTAMP  | DEFAULT NOW() | 记录创建时<br>间     |
| updated_at              | TIMESTAMP  | DEFAULT NOW() | 最后更新时<br>间     |

表 4-8 用户口味反馈表结构

| 字段名           | 数据类型         | 约束                   | 说明      |
|---------------|--------------|----------------------|---------|
| id            | INT          | PK,AUTO_INCREMENT    | 主键      |
| user_id       | INT          | FK→users.id NOT NULL | 用户 ID   |
| feedback_date | DATE         | NOT NULL             | 反馈日期    |
| satisfaction  | ENUM         | NOT NULL             | 用户满意度   |
| genres_added  | VARCHAR(255) | —                    | 新增偏好流派  |
| artists_added | VARCHAR(255) | —                    | 新增偏好艺术家 |
| created_at    | TIMESTAMP    | DEFAULT NOW()        | 记录创建时间  |

表 4-9 申诉表结构

| 字段名           | 数据类型         | 约束                | 说明                        |
|---------------|--------------|-------------------|---------------------------|
| id            | INT          | PK,AUTO_INCREMENT | 主键                        |
| username      | VARCHAR(50)  | NOT NULL          | 申诉账号名                     |
| user_id       | INT          | FK→users.id       | 关联用户 ID                   |
| appeal_type   | ENUM         | NOT NULL          | frozen/deleted            |
| reason        | TEXT         | —                 | 申诉理由                      |
| contact_email | VARCHAR(100) | —                 | 联系邮箱                      |
| status        | ENUM         | DEFAULT 'pending' | pending/approved/rejected |
| admin_reply   | TEXT         | —                 | 管理员回复内容                   |
| create_time   | TIMESTAMP    | DEFAULT NOW()     | 申诉创建时间                    |
| update_time   | TIMESTAMP    | DEFAULT NOW()     | 最后处理时间                    |

表 4-10 用户屏蔽内容表结构

| 字段名           | 数据类型         | 约束                     | 说明                 |
|---------------|--------------|------------------------|--------------------|
| id            | INT          | PK,AUTO_INCREMENT      | 主键                 |
| user_id       | INT          | FK→users.id NOT NULL   | 用户 ID              |
| block_type    | ENUM         | NOT NULL               | 屏蔽类型               |
| block_value   | VARCHAR(100) | NOT NULL               | 屏蔽的流派名或歌手名         |
| block_count   | INT          | DEFAULT 1              | 屏蔽次数               |
| blocked_at    | TIMESTAMP    | DEFAULT NOW()          | 最近一次屏蔽时间           |
| blocked_until | DATE         | NOT NULL               | 屏蔽到期日              |
| is_active     | TINYINT(1)   | DEFAULT 1              | 是否生效（1/0）          |
| song_id       | INT          | PK,FK→songs.id,NOTNULL | 歌曲 ID（主键，与歌曲主表一对一） |
| cnt_7d        | INT          | NOT NULL,DEFAULT 0     | 近 7 天播放次数          |
| cnt_30d       | INT          | NOT NULL,DEFAULT 0     | 近 30 天播放次数         |

表 4-11 歌曲滚动统计表结构

| 字段名         | 数据类型      | 约束                                           | 说明                    |
|-------------|-----------|----------------------------------------------|-----------------------|
| trending    | FLOAT     | NOT NULL,DEFAULT 0                           | 热度趋势（7 天播放量/30 天日均+1） |
| total_plays | BIGINT    | NOT NULL,DEFAULT 0                           | 累计总播放次数               |
| updated_at  | TIMESTAMP | NOT NULL,DEFAULT<br>NOW() ON UPDATE<br>NOW() | 最后更新时间                |

附表 2:

表 5-1 系统实验环境配置

| 类别         | 环境           | 配置                                    |
|------------|--------------|---------------------------------------|
| 硬件         | 操作系统         | Windows 11 专业版（64 位）                  |
|            | 处理器          | 13th Gen Intel(R) Core(TM) i9-13900HX |
|            | GPU          | NVIDIA GeForce RTX 4060               |
|            | 显存/内存        | 8 GB VRAM/16 GB RAM                   |
|            | CUDA 版本      | 13.1（驱动 591.74）                       |
| Python 端软件 | Python       | 3.12                                  |
|            | PyTorch      | 2.6.0（cu124）                          |
|            | LightGBM     | 4.6.0                                 |
|            | FAISS        | 1.13.2                                |
|            | PySpark      | 4.0.0                                 |
|            | scikit-learn | 1.5.2                                 |
|            | Pandas/NumPy | 2.2.2/1.26.4                          |

表 5-1（续表） 系统实验环境配置

| 类别         | 环境                    | 配置      |
|------------|-----------------------|---------|
| Python 端软件 | Redis                 | 7.4.0   |
|            | implicit              | 0.7.2   |
|            | SciPy                 | 1.14.1  |
| Java 端软件   | JDK（Eclipse Adoptium） | 23 LTS  |
|            | Jakarta Servlet/JSP   | 6.1/3.1 |
|            | MySQL                 | 8.4.0   |

表 5-2 C3P0 连接池配置参数

| 参数       | 取值       | 说明            |
|----------|----------|---------------|
| 初始连接数    | 5        | 启动时预建连接数      |
| 最小连接数    | 5        | 连接池保持的下限      |
| 最大连接数    | 20       | 并发扩展上限        |
| 最大空闲时间   | 300 秒    | 超时未使用的连接自动回收  |
| 获取连接超时   | 30000 毫秒 | 超时未获取到连接则抛出异常 |
| 空闲连接检测周期 | 30 秒     | 定期发送心跳探测连接存活  |
| 签出前存活验证  | 开启       | 签出连接前执行轻量查询验活 |
| 获取失败重试次数 | 3 次      | 连接获取失败后的最大重试数 |
| 重试等待间隔   | 1000 毫秒  | 每次重试前的等待时间    |

附表 3:

表 5-3 用户行为交互数据字段说明

| 字段名称               | 数据类型   | 字段说明                                          |
|--------------------|--------|-----------------------------------------------|
| msno               | String | 用户唯一标识                                        |
| song_id            | String | 歌曲唯一标识                                        |
| source_system_tab  | String | 用户触达该歌曲的功能页面                                  |
| source_system_tab  | String | 用户触达该歌曲的功能页面                                  |
| source_screen_name | String | 用户当前所在的具体界面名称                                 |
| source_type        | String | 播放入口类型                                        |
| target             | Int    | 目标变量：用户首次收听后一个<br>月内是否产生重复收听行为（1 为<br>是，0 为否） |

表 5-4 歌曲元数据字段说明

| 字段名称        | 数据类型   | 字段说明       |
|-------------|--------|------------|
| song_id     | String | 歌曲唯一标识     |
| song_length | Int    | 歌曲时长       |
| genre_ids   | String | 流派标识列表     |
| artist_name | String | 演唱艺术家名称    |
| composer    | String | 作曲者名称      |
| lyricist    | String | 作词者名称      |
| language    | Int    | 歌曲语言编码     |
| name        | String | 歌曲名称       |
| isrc        | String | 国际标准录音制品编码 |

表 5-5 用户画像数据字段说明

| 字段名称                   | 数据类型   | 字段说明    |
|------------------------|--------|---------|
| msno                   | String | 用户唯一标识  |
| city                   | Int    | 城市编码    |
| bd                     | Int    | 出生年份    |
| gender                 | String | 性别      |
| registered_via         | Int    | 注册渠道编码  |
| registration_init_time | Int    | 注册时间戳   |
| expiration_date        | Int    | 会员过期时间戳 |

表 5-6 特征工程各维度特征概览

| 特征维度     | 特征数 | 代表特征                                                         | 类型    |
|----------|-----|--------------------------------------------------------------|-------|
| 用户画像     | 10  | 性别、年龄、城市、账龄分桶、播放量对数、平均完播率、流派多样性熵、近 30 日活跃天数、收听高峰时段、用户 id     | 稀疏\稠密 |
| 歌曲属性     | 12  | 流派、语言、歌手、来源国、年份分桶、时长分桶、播放量对数、平均完播率、归一化热度、歌曲年龄对数、歌曲 id        | 稀疏\稠密 |
| 上下文      | 1   | 来源渠道（source_channel）                                         | 稀疏    |
| 交叉匹配     | 5   | 用户-流派匹配度、用户-歌手匹配度、用户-语言匹配度、用户-国家匹配度、歌单艺术家计数对数                | 稠密    |
| 时序行为     | 10  | 距上次听同曲天数、历史播放次数、近 7/30 日播放量对数、近 7 日平均完播率、歌曲近 7/30 日播放量、热度趋势比 | 稠密    |
| SVD 协同嵌入 | 26  | 用户-歌曲矩阵分解 10 维、歌曲-用户矩阵分解 10 维、用户-歌手矩阵分解 5 维、SVD 点积分数 1 维     | 稠密    |
| 行为先验     | 3   | 用户重复收听率、歌曲重复收听率、用户跳过率                                        | 稠密    |

表 5-7 BST 粗排模型超参数配置

| 超参数                      | 取值                | 说明                       |
|--------------------------|-------------------|--------------------------|
| Dropout 率                | 0.45              | Transformer 与 DNN 层共用    |
| 批量大小                     | 4096              | GPU 显存与吞吐量平衡             |
| 稀疏特征数/Embedding<br>维度    | 14/32             | 低频标识符（出现次数<3）映射<br>至未知类别 |
| Transformer 层数/注意力<br>头数 | 3/8               | 隐层维度 128，前馈维度 256        |
| DNN 隐藏层结构                | (256,128,64)      | 三层全连接，各层后接批归一化           |
| 激活函数                     | ReLU              | 各隐藏层后接批归一化               |
| 最大训练轮次                   | 40                | 轮次                       |
| 初始学习率                    | 5e-5              | Adam 优化器                 |
| LR 衰减策略                  | CosineAnnealingLR | 最小学习率下限 1e-6             |
| 早停耐心轮次                   | 7                 | 验证 AUC 连续 7 轮不提升则终止      |

表 5-8 LightGBM 精排模型超参数配置

| 超参数               | 取值    | 说明             |
|-------------------|-------|----------------|
| num_leaves        | 160   | 每棵树最大叶节点数      |
| max_depth         | 8     | 树最大深度          |
| n_estimators      | 20000 | 最大决策树棵数        |
| learning_rate     | 0.003 | 学习率            |
| min_child_samples | 1500  | 叶节点最小样本数，防止过拟合 |
| feature_fraction  | 0.75  | 每轮特征列采样比例      |
| bagging_fraction  | 0.75  | 每轮样本行采样比例      |

表 5-8（续表） LightGBM 精排模型超参数配置

| 超参数                   | 取值     | 说明                     |
|-----------------------|--------|------------------------|
| bagging_freq          | 5      | 每 5 轮执行一次行采样           |
| reg_alpha             | 1.5    | L1 正则化系数               |
| reg_lambda            | 5.0    | L2 正则化系数               |
| objective             | binary | 二分类任务                  |
| early_stopping_rounds | 800    | 验证集 AUC 连续 800 轮无提升则终止 |

表 5-9 DeepFM 精排模型超参数配置

| 超参数              | 取值                  | 说明                       |
|------------------|---------------------|--------------------------|
| DNN 隐藏层结构        | (512, 256, 128, 64) | 四层全连接，逐层降维               |
| Dropout 率        | 0.35                | 各隐藏层 Dropout，防止过拟合       |
| 批量大小             | 8192                | 充分利用 GPU 并行计算            |
| 最大训练轮次           | 60                  | 轮次                       |
| 初始学习率            | 0.002               | Adam 优化器                 |
| weight_decay     | 1e-4                | Adam 权重衰减                |
| l2_reg_embedding | 1e-4                | 嵌入层 L2 正则化               |
| LR 衰减策略          | ReduceLROnPlateau   | 耐心轮次 3，衰减系数 0.35，下限 1e-6 |
| 早停耐心轮次           | 10                  | AUC 连续 10 轮不提升则终止        |
| 混合精度训练           | AMP（FP16/FP32）      | 加速训练，降低显存占用              |